

BÁO CÁO ĐỒ ÁN MÔN HỌC

**DELIVERY MANAGEMENT -
ỨNG DỤNG QUẢN LÝ GIAO HÀNG CHO
DOANH NGHIỆP NHỎ**

Khoa : **Công Nghệ Thông Tin**

Giảng viên hướng dẫn: **ThS. Nguyễn Mạnh Hùng**

Sinh viên thực hiện:

Tên: Bùi Trịnh Gia Bảo MSSV: 2280600170 Lớp 22DTHE1

Tên: Đinh Quốc Khánh MSSV: 2386400022 Lớp 23DKHA1

Tên: Lâm Trần Tấn Phát MSSV: 2386400038 Lớp 23DKHA1

Tên: Bùi Phúc Bình MSSV: 2386400989 Lớp 23DKHA1

TP. Hồ Chí Minh, 2025

LỜI MỞ ĐẦU

Công nghệ thông tin đang phát triển nhanh chóng và trở thành nền tảng của mọi ngành công nghiệp trên toàn thế giới trong kỷ nguyên Công nghiệp 4.0. Tại Việt Nam, việc ứng dụng công nghệ vào đời sống và sản xuất đã giúp tăng năng suất lao động, hiệu quả quản lý và chất lượng dịch vụ. Việc tin học hóa các quy trình quản lý, đặc biệt là trong vận chuyển và giao nhận hàng hóa, ngày càng trở nên cần thiết để đáp ứng nhu cầu của thị trường về tính chính xác, tốc độ và minh bạch.

Nhờ sự phổ biến của các thiết bị di động như điện thoại thông minh và máy tính bảng, các ứng dụng di động ngày càng trở nên quan trọng trong việc hỗ trợ mọi người và tổ chức thực hiện các tác vụ quản lý mọi lúc, mọi nơi. Điều này thúc đẩy sự phát triển của các giải pháp quản lý giao hàng trực tuyến, mang lại lợi ích cho các công ty bằng cách tiết kiệm thời gian, cắt giảm chi phí và nâng cao sự hài lòng của khách hàng.

Nhận thấy nhu cầu đó, nhóm chúng em đã thực hiện đề tài: “Ứng dụng Quản Lý Giao Hàng”

Dưới sự hướng dẫn của Th.S Nguyễn Mạnh Hùng, nhóm đã phát triển một hệ thống hỗ trợ quản lý giao hàng từ khâu tạo đơn hàng, phân công tài xế, theo dõi trạng thái và hoàn tất giao hàng. Họ đã thực hiện điều này bằng cách vận dụng kiến thức về phân tích, thiết kế và phát triển phần mềm. Công cụ này hướng đến việc cải thiện sự phối hợp giữa công ty và tài xế, hợp lý hóa quy trình quản lý và tăng cường tính minh bạch trong hoạt động giao hàng.

Do thời gian và kiến thức của nhóm còn hạn chế, đề tài chắc chắn không tránh khỏi thiếu sót. Nhóm rất mong nhận được ý kiến đóng góp của quý thầy cô để hoàn thiện hơn trong tương lai.

Chúng em xin chân thành cảm ơn!

Sinh viên thực hiện
Bùi Trịnh Gia Bảo
Đinh Quốc Khánh
Lâm Trần Tấn Phát
Bùi Phúc Bình

LỜI CẢM ƠN

Trước hết, chúng em xin chân thành cảm ơn các thầy cô giáo Khoa Công nghệ Thông tin, Đại học Công nghệ Thành phố Hồ Chí Minh (HUTECH) đã tận tình giảng dạy, truyền đạt cho chúng em những kiến thức vô cùng quý báu và nền tảng chuyên môn vững chắc trong suốt quá trình học tập. Những kiến thức đó chính là hành trang quan trọng giúp chúng em thực hiện và hoàn thành đề tài này. Đặc biệt, chúng em xin gửi lời cảm ơn sâu sắc đến Thầy Nguyễn Mạnh Hùng, người hướng dẫn của chúng em, vì sự hỗ trợ, hướng dẫn tận tình và cung cấp những thông tin, công nghệ và kinh nghiệm thực tế cần thiết để chúng em hoàn thành đề tài. Ngoài ra, chúng tôi xin cảm ơn gia đình, anh chị em và bạn bè đã luôn động viên, khích lệ, hỗ trợ về mặt tài chính và tinh thần, giúp chúng tôi hoàn thành thành công quá trình nghiên cứu và thực hiện dự án.

TP. Hồ Chí Minh, ngày 20 tháng 11 năm 2025

LỜI CAM ĐOAN

Chúng em xin cam đoan rằng toàn bộ nội dung của Đồ án môn học “Delivery Management – Ứng dụng quản lý giao hàng cho doanh nghiệp nhỏ” là sản phẩm nghiên cứu và phát triển của chính nhóm chúng em. Những nội dung, kết quả và giải pháp được trình bày trong báo cáo này là kết quả của quá trình học tập, tìm hiểu và thực hành nghiêm túc của nhóm, dưới sự hướng dẫn của giảng viên phụ trách. Chúng em xin chịu hoàn toàn trách nhiệm về tính trung thực, chính xác của nội dung và cam kết không sao chép hoặc vi phạm bản quyền dưới bất kỳ hình thức nào.

TP. Hồ Chí Minh, ngày 29 tháng 10 năm 2025

Người cam đoan:

Bùi Trịnh Gia Bảo

Đinh Quốc Khánh

Lâm Trần Tấn Phát

Bùi Phúc Bình

1.1. KHẢO SÁT THỰC TRẠNG.....	4
1.2. TÍNH KHẢ THI CỦA BÀI TOÁN.....	5
1.3. ĐỐI TƯỢNG NGHIÊN CỨU SỬ DỤNG.....	6
1.4. MỤC TIÊU NGHIÊN CỨU.....	7
1.5. PHẠM VI GIỚI HẠN.....	7
1.6. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN CỦA ĐỀ TÀI.....	8
1.7. CẤU TRÚC ĐỒ ÁN.....	9
2.1. GIỚI THIỆU VỀ NGÔN NGỮ SỬ DỤNG.....	10
2.1.1. Giới thiệu ngôn ngữ lập trình C#.....	10
2.1.2. Giới thiệu về Dart.....	10
2.1.3. GIỚI THIỆU VỀ HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU POSTGRESQL.....	11
2.1.4. GIỚI THIỆU VỀ FLUTTER.....	11
2.2. CÔNG CỤ SỬ DỤNG.....	12
2.2.1. Giới thiệu về IDE – Visual Studio 2022.....	12
2.2.2. Giới thiệu về IDE – Visual Studio Code.....	13
2.2.3. Giới thiệu về GitHub.....	14
2.3. MÔ HÌNH VÀ KỸ THUẬT.....	14
2.3.1. Giới thiệu về API.....	14
2.3.2. GIỚI THIỆU CÁC THƯ VIỆN HỖ TRỢ.....	16
2.3.2.1. Dio.....	16
2.3.2.2. Flutter Dotenv.....	16
2.3.2.3. Shared Preferences.....	16
2.3.2.4. JWT Decoder.....	16
2.3.2.5. Flutter Map và Lat Long 2.....	17
2.3.2.6. FL Chart.....	17
2.3.2.7. Permission Handler.....	17
2.3.2.8. Speech to Text và Flutter TTS.....	17
2.3.2.9. BCrypt.Net-Next (4.0.3).....	17
2.3.2.10. Google.OrTools (9.11.4210).....	18
2.3.2.11. Microsoft.AspNetCore.Authentication.JwtBearer (8.0.8).....	18
2.3.2.12. System.IdentityModel.Tokens.Jwt (8.8.0).....	18
2.3.2.13. Microsoft.EntityFrameworkCore.SqlServer (8.0.8).....	18
2.3.2.14. Microsoft.EntityFrameworkCore.Tools (8.0.8).....	18
2.3.2.15. Swashbuckle.AspNetCore (6.6.2).....	19
3.1. MÔ TẢ NGHIỆP VỤ.....	19
3.2. MÔ HÌNH QUAN HỆ.....	22

3.2.1. Mô tả các quan hệ.....	22
3.2.2. Lược đồ quan hệ cơ sở dữ liệu.....	24
4.1. MÔ HÌNH HỆ THỐNG.....	25
4.1.1. Mô hình cấu trúc hệ thống.....	25
4.1.2. Mô hình tổng thể hệ thống.....	25
4.2. GIAO DIỆN API.....	25
5.1. KẾT LUẬN CHUNG.....	28
5.2. KẾT QUẢ ĐẠT ĐƯỢC.....	28
5.3. KẾT QUẢ CHƯA ĐẠT ĐƯỢC.....	30
5.4. HƯỚNG PHÁT TRIỂN.....	31

1. CHƯƠNG 1 TỔNG QUAN

1.1. KHẢO SÁT THỰC TRẠNG

Sự phát triển nhanh chóng của khoa học kỹ thuật và công nghệ đã làm thay đổi đáng kể thói quen sinh hoạt, làm việc và quản lý trong đời sống hiện đại. Ngày nay, thông tin và dữ liệu được xử lý, lưu trữ và truy cập dễ dàng thông qua các thiết bị thông minh, giúp con người tối ưu hóa công việc và nâng cao năng suất.

Trong lĩnh vực vận chuyển và giao nhận hàng hóa, việc tổ chức, theo dõi và quản lý đơn hàng là một nhiệm vụ quan trọng, đòi hỏi tính chính xác, minh bạch và cập nhật liên tục. Tuy nhiên, với khối lượng đơn hàng lớn mỗi ngày cùng số lượng tài xế ngày càng tăng, việc quản lý thủ công bằng giấy tờ, bảng tính hoặc liên hệ qua điện thoại thường gây mất nhiều thời gian, dễ dẫn đến sai sót, thất lạc thông tin hoặc chậm trễ trong quá trình điều phối.

Thực tế cho thấy nhiều doanh nghiệp nhỏ và vừa trong lĩnh vực giao hàng đang gặp khó khăn trong việc theo dõi tình trạng từng đơn hàng theo thời gian thực, quản lý lịch trình và hiệu suất tài xế, cũng như kiểm soát dữ liệu khách hàng, tuyến đường, chi phí và doanh thu một cách thống nhất. Một số nền tảng giao hàng trực tuyến đã xuất hiện trên thị trường nhưng đa phần có chi phí cao, không phù hợp với quy mô nhỏ hoặc không đảm bảo tính tùy biến và bảo mật dữ liệu nội bộ.

Từ những vấn đề trên, việc xây dựng một ứng dụng quản lý giao hàng riêng biệt với giao diện thân thiện, khả năng bảo mật cao và dễ triển khai là hướng đi thiết thực và cần thiết. Xuất phát từ nhu cầu thực tế này, nhóm chúng em lựa chọn đề tài “*Ứng dụng Quản Lý Giao Hàng*”.

Đề tài hướng đến việc phát triển một hệ thống giúp doanh nghiệp, tài xế và khách hàng dễ dàng theo dõi quy trình giao hàng từ khâu tạo đơn đến khi hoàn tất giao nhận, đồng thời tối ưu hóa việc quản lý dữ liệu và giám sát hiệu quả hoạt động. Hệ thống này góp phần nâng cao hiệu quả điều phối, giảm thiểu sai sót và thể hiện xu hướng chuyển đổi số trong lĩnh vực logistics, phù hợp với định hướng phát triển kinh tế số tại Việt Nam hiện nay

1.2. TÍNH KHẢ THI CỦA BÀI TOÁN

Từ việc khảo sát thực trạng hoạt động của các doanh nghiệp vừa và nhỏ trong lĩnh vực vận chuyển, nhóm nhận thấy việc xây dựng một **hệ thống quản lý giao hàng và đội xe** là hoàn toàn khả thi, đồng thời mang lại nhiều lợi ích thiết thực. Cụ thể, hệ thống có thể giải quyết được các vấn đề sau:

a. Về mặt quản lý phương tiện và tài xế:

Hệ thống cho phép chủ doanh nghiệp quản lý thông tin chi tiết của từng xe (biển số, loại xe, tình trạng bảo dưỡng, giấy tờ xe, lịch bảo trì, v.v.) và thông tin tài xế (họ tên, số điện thoại, lịch làm việc, đơn hàng được giao). Nhờ đó, chủ doanh nghiệp có thể dễ dàng giám sát đội xe và phân công công việc hiệu quả hơn.

b. Về mặt quản lý đơn hàng và hàng hóa:

Hệ thống giúp chủ doanh nghiệp tạo, chỉnh sửa, xóa và theo dõi từng đơn hàng. Mỗi đơn hàng bao gồm danh sách hàng hóa, số lượng, địa điểm giao, thời gian và các ghi chú đặc biệt (như “gọi trước 15 phút”, “hàng dễ vỡ”,...). Việc quản lý tập trung giúp tránh sai sót, trùng lặp và tối ưu hóa quy trình xử lý đơn hàng.

c. Về mặt điều phối và tối ưu lộ trình giao hàng:

Ứng dụng hỗ trợ tài xế xem các điểm giao hàng theo thứ tự lộ trình đã được tối ưu. Hệ thống có khả năng tự động sắp xếp tuyến đường hợp lý để tiết kiệm thời gian và chi phí nhiên liệu. Khi tài xế hoàn thành giao hàng tại mỗi điểm, chủ doanh nghiệp sẽ nhận được thông báo cập nhật theo thời gian thực.

d. Về mặt triển khai và công nghệ:

Hệ thống được phát triển theo mô hình **Client–Server**, với **Frontend sử dụng Flutter** để xây dựng giao diện nền tảng Android, và **Backend phát triển bằng ASP.NET Core C#** để cung cấp các API RESTful kết nối với **SQL Server**. Nhờ đó, việc triển khai hệ thống trên nhiều thiết bị và nền tảng là hoàn toàn khả thi, đảm bảo hiệu suất và tính ổn định cao.

Từ các yếu tố trên, nhóm quyết định tập trung phát triển **cơ sở dữ liệu quản lý đơn hàng, xe và tài xế**, đồng thời xây dựng **ứng dụng di động dành cho tài xế** và **trang quản trị dành cho chủ doanh nghiệp**. Mục tiêu là tạo ra một hệ thống có khả năng hoạt động ổn định, dễ mở rộng, bảo mật dữ liệu tốt, và đáp ứng được nhu cầu thực tế của các doanh nghiệp giao hàng quy mô nhỏ.

1.3. ĐỐI TƯỢNG NGHIÊN CỨU SỬ DỤNG

Nắm bắt được nhu cầu quản lý và vận hành trong các doanh nghiệp nhỏ hoạt động trong lĩnh vực vận chuyển, nhóm chúng em nhận thấy việc phát triển một **hệ thống quản lý giao hàng và tối ưu lộ trình vận chuyển (Delivery Management System)** là cần thiết và khả thi. Hệ thống này cho phép người quản lý, tài xế và các bộ phận liên quan phối hợp hiệu quả trong toàn bộ quy trình giao nhận.

a. Về phía **chủ doanh nghiệp**, hệ thống cung cấp một cổng quản trị (Admin Portal) giúp theo dõi,

thống kê và quản lý toàn bộ thông tin về xe, tài xế, đơn hàng, điểm giao, và hàng hóa. Chủ có thể theo dõi tình trạng giao hàng theo thời gian thực, nhận thông báo khi đơn hoàn tất, và xem báo cáo tổng hợp để đưa ra quyết định nhanh chóng và chính xác.

b. Về phía **tài xế**, ứng dụng di động được cài đặt trên điện thoại đặt trong xe giúp hiển thị danh sách các điểm giao hàng trong ngày, sắp xếp theo lộ trình đã được tối ưu. Tài xế có thể xem chi tiết hàng hóa, ghi chú tuyến đường, xác nhận khi đã đến nơi và khi hoàn tất giao hàng. Các thông tin này được đồng bộ trực tiếp với hệ thống trung tâm để cập nhật trạng thái đơn hàng cho chủ doanh nghiệp.

c. Về phía **bộ phận kỹ thuật hoặc quản trị hệ thống**, họ có nhiệm vụ bảo trì, cập nhật và đảm bảo tính ổn định, bảo mật dữ liệu trong toàn bộ hệ thống. Bộ phận này cũng chịu trách nhiệm theo dõi hiệu năng của ứng dụng và thực hiện các cải tiến khi cần thiết.

d. Hệ thống được phát triển dựa trên mô hình Client–Server, trong đó **Frontend** sử dụng **Flutter** để xây dựng giao diện người dùng trên đa nền tảng (Android, iOS, Web), còn **Backend** sử dụng **ASP.NET Core C#** kết nối với **SQL Server** thông qua API RESTful. Mô hình này đảm bảo hệ thống dễ mở rộng, dễ bảo trì và triển khai hiệu quả trên nhiều môi trường khác nhau.

Từ các đối tượng trên, nhóm chúng em tập trung phát triển hai phần chính: **ứng dụng di động dành cho tài xế** và **cổng quản trị dành cho chủ doanh nghiệp**, hướng đến một giải pháp có tính ổn định, dễ sử dụng, đáp ứng nhu cầu thực tế của các doanh nghiệp vừa và nhỏ trong hoạt động giao hàng.

1.4. MỤC TIÊU NGHIÊN CỨU

BackEnd: Nghiên cứu và xây dựng hệ thống API theo kiến trúc phân lớp (layered architecture) sử dụng ASP.NET Core và ngôn ngữ lập trình C#. Vận dụng Entity Framework Core để làm việc với cơ sở dữ liệu và triển khai các nghiệp vụ cốt lõi như quản lý đơn hàng, tài xế, và phương tiện.

FrontEnd: Nghiên cứu vận dụng Flutter với ngôn ngữ Dart để phát triển ứng dụng di động (mobile app) đa nền tảng. Ứng dụng được thiết kế với giao diện cho hai vai trò chính:

Quản lý (Owner): Giao diện quản lý, giám sát đơn hàng, tài xế, sản phẩm và các điểm giao nhận.

Tài xế (Driver): Giao diện nhận đơn hàng, xem thông tin chi tiết và theo dõi lộ trình giao hàng đã được tối ưu.

Cơ Sở Dữ Liệu: Vận dụng PostgreSQL – một hệ quản trị cơ sở dữ liệu mã nguồn mở – để quản lý

dữ liệu tập trung cho hệ thống API, kết nối thông qua Entity Framework Core.

Công nghệ & Thư viện:

Tích hợp dịch vụ tối ưu hóa tuyến đường (Route Optimization) để tính toán và cung cấp lộ trình giao hàng hiệu quả nhất cho tài xế.

Tích hợp các mô hình AI tạo sinh (Gemini, OpenAI) để xây dựng tính năng chatbot, tư vấn và hỗ trợ người dùng.

Vận dụng các dịch vụ Speech-to-Text (nhận diện giọng nói) và Text-to-Speech (tổng hợp giọng nói) để tăng cường khả năng tương tác và trải nghiệm người dùng trên ứng dụng di động.

Quản lý dự án: Sử dụng GitHub để quản lý source code, theo dõi các công việc và cộng tác giữa các thành viên trong dự án.

1.5. PHẠM VI GIỚI HẠN

Phạm vi giới hạn của đề án này sẽ tập trung vào việc phát triển Hệ thống Quản lý Vận tải cho doanh nghiệp. Hệ thống bao gồm một BackEnd API (xây dựng bằng ASP.NET Core) để xử lý nghiệp vụ và một Ứng dụng Di động (xây dựng bằng Flutter) cho hai đối tượng người dùng chính: Quản lý (Owner) và Tài xế (Driver).

Đề án sẽ không bao gồm các tính năng phức tạp ở quy mô lớn như: tích hợp cổng thanh toán trực tuyến, quản lý kho bãi (WMS) toàn diện, hay tích hợp với các hệ thống của bên thứ ba (3PL, các sàn thương mại điện tử). Chức năng theo dõi vị trí sẽ tập trung vào việc tối ưu hóa lộ trình, không phải là một hệ thống giám sát GPS thời gian thực (real-time) với độ chính xác cao.

Phạm vi cũng sẽ tập trung chính vào việc phát triển Ứng dụng Di động (Flutter), với mục tiêu cung cấp một nền tảng thuận tiện cho Quản lý (quản lý đơn hàng, tài xế, phương tiện) và Tài xế (nhận đơn, xem lộ trình đã được tối ưu). Phạm vi chưa mở rộng đến việc tích hợp sâu vào hệ thống kế toán, quản lý nhân sự, hay trở thành một hệ thống hoạch định nguồn lực doanh nghiệp (ERP) toàn diện.

1.6. Ý NGHĨA KHOA HỌC VÀ THỰC TIỄN CỦA ĐỀ TÀI

Xây dựng hệ thống quản lý giao hàng và tối ưu lộ trình vận chuyển cho doanh nghiệp nhỏ là một đề tài mang ý nghĩa khoa học và thực tiễn quan trọng. Hệ thống hướng đến việc cung cấp một nền

tăng công nghệ hiện đại, giúp doanh nghiệp quản lý phương tiện, tài xế, đơn hàng và lộ trình giao hàng một cách hiệu quả, tiết kiệm thời gian và chi phí vận hành.

Ý nghĩa khoa học của đề tài là việc áp dụng các kiến thức và công nghệ trong lĩnh vực phát triển phần mềm, cơ sở dữ liệu và tối ưu hóa tuyến đường để giải quyết bài toán thực tế trong quản lý giao hàng. Đề tài vận dụng các công nghệ hiện đại như Flutter cho giao diện người dùng đa nền tảng, ASP.NET Core C# cho dịch vụ web và SQL Server cho hệ quản trị cơ sở dữ liệu, góp phần nâng cao khả năng ứng dụng của sinh viên trong phát triển hệ thống thực tế.

Thực tiễn của đề tài thể hiện qua việc cung cấp một giải pháp quản lý tập trung và đồng bộ giữa chủ doanh nghiệp và tài xế. Ứng dụng giúp chủ doanh nghiệp theo dõi trạng thái giao hàng theo thời gian thực, quản lý xe và tài xế, trong khi tài xế có thể nhận đơn, xem lộ trình tối ưu và xác nhận giao hàng nhanh chóng. Điều này giúp nâng cao hiệu quả vận hành, giảm sai sót, tăng độ tin cậy và tính chuyên nghiệp trong dịch vụ giao hàng.

Tổng quan, đề tài Ứng dụng quản lý giao hàng và tối ưu lộ trình không chỉ có ý nghĩa trong việc đưa công nghệ thông tin vào tự động hóa quy trình logistics mà còn góp phần thúc đẩy chuyển đổi số trong các doanh nghiệp nhỏ tại Việt Nam, mở ra hướng phát triển bền vững và hiện đại hơn cho lĩnh vực vận tải và thương mại điện tử.

1.7. CẤU TRÚC ĐỒ ÁN

Đồ án Delivery Management – Ứng dụng quản lý giao hàng cho doanh nghiệp nhỏ tập trung vào các quá trình nghiên cứu và phát triển như sau:

Về việc phát triển cơ sở dữ liệu:

Cơ sở dữ liệu được thiết kế nhằm phản ánh đầy đủ mối quan hệ giữa các thực thể chính trong hệ thống như xe, tài xế, đơn hàng, hàng hóa, điểm giao và tuyến đường. Việc xây dựng cơ sở dữ liệu đảm bảo tính toàn vẹn, tránh dư thừa thông tin và hỗ trợ truy xuất nhanh. Cơ sở dữ liệu được triển khai trên SQL Server, sử dụng các ràng buộc khóa chính – khóa ngoại và chỉ mục để đảm bảo tính chính xác và hiệu quả trong quản lý dữ liệu.

Về việc phát triển API:

API được xây dựng bằng ASP.NET Core (C#) nhằm cung cấp dịch vụ trung gian giữa cơ sở dữ liệu và ứng dụng người dùng. Hệ thống được phát triển theo mô hình RESTful API, gồm các nhóm chức năng chính như: quản lý tài xế, quản lý xe, quản lý đơn hàng, quản lý

hàng hóa, điểm giao và tối ưu tuyến đường. API hỗ trợ xác thực người dùng bằng JWT, phân quyền truy cập, xử lý nghiệp vụ tại tầng Service và giao tiếp với cơ sở dữ liệu thông qua Entity Framework Core. Toàn bộ dịch vụ được thiết kế đảm bảo khả năng mở rộng, bảo mật và tối ưu hiệu năng trong quá trình xử lý dữ liệu.

Về việc hệ thống:

Delivery Management – Admin Portal:

Hệ thống quản trị được phát triển trên nền tảng Website theo mô hình MVC đơn giản, cho phép chủ doanh nghiệp hoặc người quản lý truy cập để quản lý thông tin xe, tài xế, hàng hóa, đơn hàng và điểm giao. Giao diện hỗ trợ thao tác thêm, sửa, xóa, tìm kiếm và thống kê dữ liệu. Ngoài ra, người quản lý có thể theo dõi trạng thái giao hàng theo thời gian thực và nhận thông báo khi đơn hàng được hoàn thành.

Delivery Management – Driver App:

Ứng dụng được phát triển bằng Flutter, triển khai trên nền tảng Mobile nhằm hỗ trợ tài xế trong việc xem danh sách các điểm giao hàng trong ngày, chi tiết từng đơn hàng và tuyến đường được hệ thống tối ưu sẵn. Ứng dụng cho phép tài xế xác nhận khi đến nơi, hoàn tất giao hàng và gửi thông tin cập nhật về hệ thống trung tâm. Giao diện được thiết kế thân thiện, dễ sử dụng và đảm bảo khả năng hoạt động ổn định trong suốt quá trình vận hành.

Tổng thể, đồ án được phát triển theo mô hình Client–Server, trong đó Backend chịu trách nhiệm xử lý nghiệp vụ và quản lý dữ liệu, còn Frontend đảm nhận hiển thị, tương tác người dùng và giao tiếp thông qua API. Hệ thống đáp ứng yêu cầu thực tế về quản lý giao hàng, tối ưu lộ trình và nâng cao hiệu quả vận hành cho các doanh nghiệp nhỏ.

2. CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1. GIỚI THIỆU VỀ NGÔN NGỮ SỬ DỤNG

2.1.1. GIỚI THIỆU NGÔN NGỮ LẬP TRÌNH C#

C# (hay C sharp) là một ngôn ngữ lập trình đơn giản, được phát triển bởi đội ngũ kỹ sư của Microsoft vào năm 2000. C# là ngôn ngữ lập trình hiện đại, hướng đối tượng và được xây dựng trên nền tảng của hai ngôn ngữ C++ và Java .

Đặc trưng cơ bản của C# :

Đơn giản: Loại bỏ những vấn đề phức tạp đã có trong Java và C++ như tính đa kế thừa, lớp cơ sở ảo...

Hiện đại: Xử lý ngoại lệ, tự động trong thu gom bộ nhớ, bảo mật mã nguồn, dữ liệu mở rộng...

Hướng đối tượng: Sở hữu cả 4 tính chất quan trọng, đặc trưng là tính kế thừa, tính đóng gói, tính trừu tượng và tính đa hình.

Ít từ khoá: Từ khoá chỉ được dùng nhằm mục đích mô tả thông tin nhưng lập trình viên vẫn có thể sử dụng nó để thực hiện mọi nhiệm vụ.

Mã nguồn mở: Được phát triển, điều hành một cách độc lập với Microsoft. - Đa nền tảng: Có thể hoạt động tốt trên nhiều nền tảng như Windows, Linux và Mac. - Tiến hoá: C# vẫn đang được nâng cấp và cho ra mắt các phiên bản mới với nhiều tính năng vượt trội và khả năng làm việc mạnh mẽ hơn. Hiện C# có thể làm việc với console, điện toán đám mây, phần mềm học máy...

Ứng dụng của C# : Ngôn ngữ này có ứng dụng trên Windows, Web và các thành phần, điều khiển.

Trên Windows: C# với framework .NET được dùng để tạo ra các ứng dụng trên Windows như Microsoft Office, Visual Studio, Skype, Photoshop... - Trên Web: C# hỗ trợ lập trình viên tạo các ứng dụng Web nhờ sự hỗ trợ của ASP.NET. Với ngôn ngữ này, các ứng dụng có thể chạy mượt mà trên máy chủ. - Thành phần, điều khiển: C# còn được ứng dụng trong xây dựng nhiều thành phần của máy chủ. Đây là một trong các ứng dụng quan trọng của ngôn ngữ lập trình C#.

2.1.2. GIỚI THIỆU VỀ DART

Dart là ngôn ngữ lập trình cho Flutter, bộ công cụ giao diện người dùng của Google để xây dựng các ứng dụng Mobile, Web và Desktop app đơn giản, bắt mắt người dùng, được biên dịch nguyên bản từ một cơ sở mã code duy nhất. **Các tính năng của ngôn ngữ Dart:**

Hướng đối tượng: Dart sử dụng dữ liệu dưới dạng đối tượng thay vì coi dữ liệu là hàm hoặc logic và hỗ trợ cả các khái niệm lập trình hướng đối tượng cơ bản. Không đồng bộ: Dart không có tính đồng bộ nhưng cho phép đồng thời nâng cao. Nghĩa là có thể chạy đồng thời nhiều tác vụ với Dart bằng cách sử dụng thể độc lập (phân lập).

Các thư viện tích hợp: Dart bao gồm các thư viện tích hợp mở rộng như Input – Output (IO), Software Development Kit (SDK)... Ta có thể tìm các đoạn mã code viết sẵn trong những thư viện này và tối ưu theo mục đích của mình.

Hỗ trợ đa nền tảng: Dart có thể hoạt động trên nhiều hệ điều hành khác nhau Windows, Linux, macOS cũng như nhiều hệ điều hành khác bởi tính năng Máy ảo Dart.

2.1.3. GIỚI THIỆU VỀ HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU SQLSERVER

Microsoft SQL Server là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mạnh mẽ do Microsoft phát triển, được sử dụng rộng rãi trong các hệ thống doanh nghiệp và ứng dụng web. SQL Server cung cấp khả năng lưu trữ, quản lý và truy xuất dữ liệu hiệu quả, hỗ trợ toàn diện cho các tính năng như khóa ngoại, ràng buộc toàn vẹn, giao dịch, stored procedures, views, triggers, và các truy vấn phức tạp.

Với kiến trúc tối ưu cho hiệu năng và độ tin cậy cao, SQL Server là lựa chọn phổ biến trong các môi trường sản xuất yêu cầu tính ổn định, bảo mật, và khả năng mở rộng. Hệ quản trị này có thể xử lý khối lượng dữ liệu lớn, hỗ trợ phân tích dữ liệu, sao lưu – phục hồi tự động và bảo mật cấp doanh nghiệp. SQL Server còn tích hợp tốt với các công nghệ .NET, giúp quá trình phát triển ứng dụng backend bằng ASP.NET Core trở nên thuận lợi và nhất quán.

Công cụ SQL Server Management Studio (SSMS) là phần mềm quản trị chính thức do Microsoft cung cấp, cho phép người dùng quản lý cơ sở dữ liệu SQL Server thông qua giao diện đồ họa thân thiện. SSMS hỗ trợ đầy đủ các thao tác như tạo, chỉnh sửa, xóa bảng; viết và thực thi truy vấn SQL; xem dữ liệu; thiết lập quan hệ và phân quyền người dùng. Ngoài ra, SSMS còn cung cấp các công cụ theo dõi hiệu năng, kiểm tra log, và tối ưu câu lệnh truy vấn giúp quản trị viên vận hành hệ thống dễ dàng và hiệu quả hơn.

Phiên bản mới của SQL Server và SSMS hiện nay đã được tối ưu đáng kể về tốc độ truy vấn, bảo mật, và trải nghiệm người dùng, đồng thời hỗ trợ tốt các môi trường điện toán đám mây (Azure SQL Database) và kết nối đa nền tảng.

2.1.4. GIỚI THIỆU VỀ FLUTTER

Flutter là một khung nguồn mở do Google phát triển và hỗ trợ. Các nhà phát triển frontend và full stack sử dụng Flutter để xây dựng giao diện người dùng (UI) của ứng dụng cho nhiều nền tảng chỉ với một nền mã duy nhất.

Tại thời điểm ra mắt vào năm 2018, Flutter chủ yếu hỗ trợ phát triển ứng dụng di động. Hiện nay, Flutter hỗ trợ phát triển ứng dụng trên sáu nền tảng: iOS, Android, web, Windows, MacOS và Linux.

Ưu điểm của Flutter:

Hiệu suất gần với phát triển ứng dụng gốc. Flutter sử dụng ngôn ngữ lập trình Dart và biên dịch thành mã máy. Các thiết bị máy chủ hiểu được mã này, điều này đảm bảo hiệu suất nhanh và hiệu quả.

Kết xuất nhanh, nhất quán và có thể tùy chỉnh. Thay vì dựa vào các công cụ kết xuất theo nền tảng, Flutter sử dụng thư viện đồ họa Skia nguồn mở của Google để kết

xuất UI. Điều này mang đến cho người dùng phương tiện trực quan nhất quán cho dù họ sử dụng nền tảng nào để truy cập ứng dụng.

Công cụ thân thiện với nhà phát triển. Google đã xây dựng Flutter chú trọng vào tính dễ sử dụng. Với các công cụ như tải lại nóng, nhà phát triển có thể xem trước các thay đổi mã sẽ như thế nào mà không bị mất trạng thái. Các công cụ khác như widget inspector giúp dễ dàng trực quan hóa và giải quyết các vấn đề với bộ cục UI.

2.2. CÔNG CỤ SỬ DỤNG

2.2.1. GIỚI THIỆU VỀ IDE – VISUAL STUDIO 2022

Visual studio là một công cụ quen thuộc đối với các lập trình viên chuyên nghiệp, đặc biệt là những người lập trình theo hướng VB+ và C#. Visual Studio là hệ thống tập hợp tất cả những gì liên quan tới phát triển ứng dụng, bao gồm trình chỉnh sửa mã, trình thiết kế, gỡ lỗi. Tức là, chúng ta có thể viết code, sửa lỗi, chỉnh sửa thiết kế ứng dụng dễ dàng chỉ với 1 phần mềm Visual Studio. Không dừng lại ở đó, người dùng còn có thể

thiết kế giao diện, trải nghiệm trong Visual Studio như khi phát triển ứng dụng Xamarin, UWP bằng XAML hay Blend.

Visual Studio 2022 (theo [9]) sẽ là một ứng dụng 64-bit, không còn giới hạn ở ~ 4GB bộ nhớ trong process chính devenv.exe. Với Visual Studio 64-bit trên Windows, bạn có thể mở, chỉnh sửa, chạy và gỡ lỗi ngay cả với những giải pháp lớn nhất và phức tạp nhất mà không bị hết bộ nhớ.

Visual Studio 2022 có thể phát triển các ứng dụng hiện đại:

.NET

Visual Studio 2022 hỗ trợ đầy đủ cho .NET 6 và framework hợp nhất dành cho web, ứng dụng khách, ứng dụng di động cho cả nhà phát triển Windows và Mac. Bao gồm Giao diện người dùng Ứng dụng đa nền tảng .NET (.NET MAUI) cho các ứng dụng khách đa hệ điều hành chạy trên Windows, Android, macOS và iOS. Bạn cũng có thể sử dụng công nghệ web ASP.NET Blazor để viết ứng dụng trên máy tính để bàn thông qua .NET MAUI.

Với hầu hết các loại ứng dụng như web, máy tính để bàn và thiết bị di động, bạn có thể sử dụng

.NET Hot Reload để áp dụng các thay đổi mã mà không cần khởi động lại hoặc mất trạng thái ứng dụng.

C++

Visual Studio 2022 sẽ hỗ trợ mạnh mẽ cho khả năng xử lý của C++, với các tính năng năng suất mới, công cụ C++ 20 và IntelliSense. Các tính năng ngôn ngữ C++ 20 mới sẽ đơn giản hóa việc quản lý các cơ sở mã lớn, khả năng phân tích được cải thiện sẽ giúp các vấn đề khó khăn được gỡ lỗi dễ dàng hơn bằng các mẫu tạo sẵn và khái niệm.

Chúng tôi cũng sẽ tích hợp hỗ trợ CMake, Linux và WSL giúp bạn tạo, chỉnh sửa, xây dựng và gỡ lỗi các ứng dụng đa nền dễ dàng hơn. Nếu bạn muốn nâng cấp lên Visual Studio 2022 nhưng lo lắng về khả năng tương thích, thì khả năng tương thích nhị phân với runtime của C++ sẽ giúp bạn dễ dàng.

2.2.2. GIỚI THIỆU VỀ IDE – VISUAL STUDIO CODE

Visual Studio Code (theo [10]) chính là ứng dụng cho phép biên tập, soạn thảo các đoạn code để hỗ trợ trong quá trình thực hiện xây dựng, thiết kế website một cách nhanh chóng. Visual Studio Code hay còn được viết tắt là VS Code. Trình soạn thảo này vận hành mượt mà trên các nền tảng như Windows, macOS, Linux. Hơn thế nữa, VS Code còn cho khả năng tương thích với những thiết bị máy tính có cấu hình tầm trung vẫn có thể sử dụng dễ dàng.

Ưu điểm nổi bật của Visual Studio Code:

Đa dạng ngôn ngữ lập trình giúp người dùng thỏa sức sáng tạo và sử dụng như HTML, CSS, JavaScript, C++...

Ngôn ngữ, giao diện tối giản, thân thiện, giúp các lập trình viên dễ dàng định hình nội dung.

Các tiện ích mở rộng rất đa dạng và phong phú.

Tích hợp các tính năng quan trọng như tính năng bảo mật (Git), khả năng tăng tốc xử lý vòng lặp (Debug)...

Đơn giản hóa việc tìm quản lý hết tất cả các Code có trên hệ thống.

2.2.3. GIỚI THIỆU VỀ GITHUB

GitHub (theo [11]) là một dịch vụ nổi tiếng cung cấp kho lưu trữ mã nguồn Git cho các dự án phần mềm. Github có đầy đủ những tính năng của Git, ngoài ra nó còn bổ sung những tính năng về social để các developer tương tác với nhau. *Vài thông tin về GIT:*

Là công cụ giúp quản lý source code tổ chức theo dạng dữ liệu phân tán. - Giúp đồng bộ source code của team lên 1 server.

Hỗ trợ các thao tác kiểm tra source code trong quá trình làm việc (diff, check modifications, show history, merge source, ...)

GitHub có 2 phiên bản: miễn phí và trả phí. Với phiên bản có phí thường được các doanh nghiệp sử dụng để tăng khả năng quản lý team cũng như phân quyền bảo mật dự án. Còn lại thì phần lớn chúng ta đều sử dụng Github với tài khoản miễn phí để lưu trữ source code.

Github cung cấp các tính năng social networking như feeds, followers, và network graph để các developer học hỏi kinh nghiệm của nhau thông qua lịch sử commit. Nếu một comment để mô tả và giải thích một đoạn code. Thì với Github, commit message chính là phần mô tả hành động mà bạn thực hiện trên source code. Github trở thành một yếu tố có sức ảnh hưởng lớn trong cộng đồng nguồn mở. Cùng với Linkedin, Github được coi là một sự thay thế cho CV của bạn. Các nhà tuyển dụng cũng rất hay tham khảo Github profile để hiểu về năng lực coding của ứng viên. Giờ đây, kỹ năng sử dụng git và Github từ chỗ ưa thích sang bắt buộc phải có đối với các ứng viên đi xin việc.

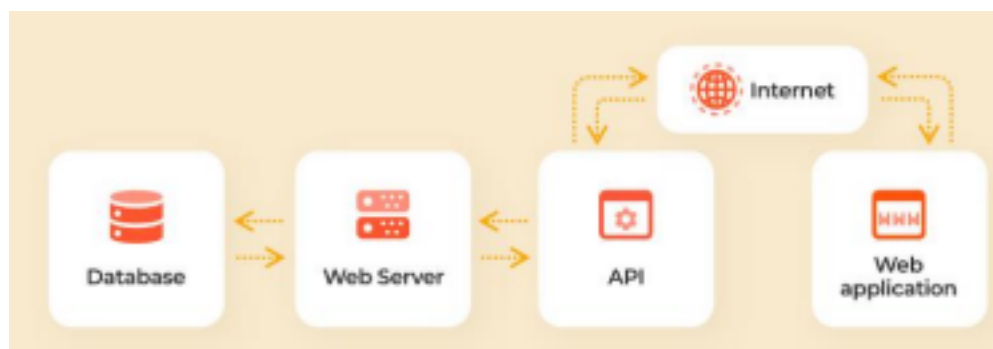
2.3. MÔ HÌNH VÀ KỸ THUẬT

2.3.1. GIỚI THIỆU VỀ API

Theo [14] tổng hợp các kiến thức, tài liệu như sau:

Khái niệm API: API là viết tắt của Application Programming Interface – phương thức trung gian kết nối các ứng dụng và thư viện khác nhau. Nó cung cấp khả năng truy xuất đến một tập các hàm hay dùng, từ đó có thể trao đổi dữ liệu giữa các ứng dụng. Thi thoảng vẫn có người lầm tưởng API là một ngôn ngữ lập trình nhưng thực ra, API chỉ là các hàm hay thủ tục thông thường. Các hàm này được viết trên nhiều ngôn ngữ lập trình khác nhau.

Đặc điểm nổi bật của API:



Hình 2.1 Quy trình hoạt động API

API sử dụng mã nguồn mở, dùng được với mọi client hỗ trợ XML, JSON. - API có khả năng đáp ứng đầy đủ các thành phần HTTP: URI, request/response headers, caching, versioning, content format... Bạn có thể sử dụng các host nằm trong phần ứng dụng hoặc trên IIS.

Mô hình web API dùng để hỗ trợ MVC như: unit test, injection, ioc container, model binder,

action result, filter, routing, controller. Ngoài ra, nó cũng hỗ trợ RESTful đầy đủ các phương thức như: GET, POST, PUT, DELETE các dữ liệu.

Được đánh giá là một trong những kiểu kiến trúc hỗ trợ tốt nhất với các thiết bị có lượng băng thông bị giới hạn như smartphone, tablet...

Ứng dụng của API:

Web API: Là hệ thống API được sử dụng trong các hệ thống website, chẳng hạn: Google, Facebook... Hầu hết các website đều cung cấp hệ thống API cho phép bạn kết nối, lấy dữ liệu hoặc cập nhật cơ sở dữ liệu. Đa số Web API được thiết kế theo tiêu chuẩn RESTful.

API trên hệ điều hành: Windows hay Linux có rất nhiều API. Họ cung cấp các tài liệu API là đặc tả các hàm, phương thức cũng như các giao thức kết nối. Nó giúp lập trình viên có thể tạo ra các phần mềm ứng dụng có thể tương tác trực tiếp với hệ điều hành. - API của thư viện phần mềm (framework): API mô tả và quy định các hành động mong muốn mà các thư viện cung cấp. Một API có thể có nhiều cách triển khai khác nhau, giúp cho một chương trình viết bằng ngôn ngữ này có thể sử dụng được thư viện viết bằng ngôn ngữ khác.

Ưu điểm:

Giao tiếp hai chiều phải được xác nhận trong các giao dịch sử dụng API. Cũng chính vì vậy mà các thông tin rất đáng tin cậy.

API là công cụ mã nguồn mở, có thể kết nối mọi lúc nhờ vào Internet. - Hỗ trợ chức năng RESTful một cách đầy đủ.

Cấu hình đơn giản khi được so sánh với WCF (Window Communication Foundation). Cung cấp cấp trải nghiệm thân thiện với người dùng.

Nhược điểm:

Tốn nhiều chi phí phát triển, vận hành, chỉnh sửa.

Đòi hỏi kiến thức chuyên sâu.

Có thể gặp vấn đề bảo mật khi bị tấn công hệ thống.

2.3.2. GIỚI THIỆU CÁC THƯ VIỆN HỖ TRỢ

2.3.2.1. Dio

Dio là một thư viện HTTP client mạnh mẽ cho Flutter và Dart, hỗ trợ đầy đủ các tính năng như interceptors, form data, request cancellation, timeout và custom configuration. Trong dự án này, Dio được sử dụng để thực hiện các yêu cầu đến API của hệ thống giao hàng, bao gồm việc gửi, nhận, và xử lý dữ liệu JSON giữa client và server. Thông qua Dio, việc quản lý kết nối mạng trở nên ổn định, linh hoạt và dễ dàng mở rộng. Ngoài ra, Dio còn hỗ trợ bắt lỗi và xử lý phản hồi một

cách trực quan, giúp đảm bảo quá trình trao đổi dữ liệu diễn ra an toàn và chính xác.

2.3.2.2. FLUTTER DOTENV

Flutter Dotsenv là thư viện hỗ trợ đọc các biến môi trường từ tệp .env trong dự án. Việc sử dụng thư viện này giúp tách biệt cấu hình hệ thống như đường dẫn API, khóa bảo mật hoặc thông tin máy chủ ra khỏi mã nguồn. Điều này giúp tăng tính bảo mật, dễ dàng quản lý môi trường phát triển và triển khai (development, staging, production) của ứng dụng quản lý giao hàng.

2.3.2.3. SHARED PREFERENCES

Shared Preferences là một thư viện cho phép lưu trữ dữ liệu đơn giản dưới dạng cặp khóa–giá trị (key–value) trên thiết bị di động. Trong ứng dụng quản lý giao hàng, thư viện này được dùng để lưu thông tin người dùng, token đăng nhập, hoặc các cài đặt tạm thời của ứng dụng. Việc lưu trữ cục bộ giúp tăng tốc độ truy cập và giảm tần suất gửi yêu cầu đến server.

2.3.2.4. JWT DECODER

JWT Decoder được sử dụng để giải mã các token JWT (JSON Web Token) được cấp khi người dùng đăng nhập. Thông qua thư viện này, ứng dụng có thể kiểm tra thời gian hết hạn, xác minh quyền truy cập và trích xuất các thông tin quan trọng trong token. Việc tích hợp JWT Decoder đảm bảo tính bảo mật và kiểm soát truy cập chặt chẽ giữa các vai trò như quản trị viên, nhân viên và tài xế.

2.3.2.5. FLUTTER MAP VÀ LAT LONG 2

Hai thư viện này được sử dụng kết hợp để hiển thị và xử lý bản đồ trong ứng dụng. Flutter Map là một plugin mã nguồn mở cung cấp khả năng hiển thị bản đồ dựa trên dữ liệu từ OpenStreetMap, còn Latlong2 hỗ trợ các phép toán liên quan đến kinh độ và vĩ độ. Trong dự án, các thư viện này giúp người dùng xem vị trí, tuyến đường giao hàng và xác định khoảng cách giữa các điểm giao nhận hàng.

2.3.2.6. FL CHART

FL Chart là thư viện hỗ trợ hiển thị biểu đồ tương tác trên Flutter. Trong hệ thống quản lý giao hàng, thư viện này được sử dụng để minh họa dữ liệu thống kê như số lượng đơn hàng, hiệu suất tài xế hoặc doanh thu theo thời gian. Việc hiển thị trực quan giúp người quản lý dễ dàng nắm bắt và phân tích dữ liệu.

2.3.2.7. PERMISSION HANDLER

Permission Handler là thư viện hỗ trợ kiểm tra và yêu cầu quyền truy cập của người dùng đối với các tính năng như vị trí, micro hoặc bộ nhớ. Trong ứng dụng giao hàng, nó được dùng để xin quyền truy cập vị trí nhằm xác định lộ trình và cập nhật trạng thái đơn hàng theo thời gian thực.

2.3.2.8. SPEECH TO TEXT VÀ FLUTTER TTS

Speech to Text được dùng để nhận diện giọng nói và chuyển đổi thành văn bản, trong khi Flutter TTS (Text to Speech) thực hiện chức năng ngược lại – chuyển đổi văn bản thành giọng nói. Sự kết hợp của hai thư viện này giúp ứng dụng hỗ trợ người dùng tương tác bằng giọng nói, ví dụ đọc to thông tin đơn hàng hoặc cho phép tải xé nhập liệu bằng lời nói trong quá trình giao nhận.

2.3.2.9. BCrypt.NET-NEXT (4.0.3)

BCrypt.Net-Next là thư viện cung cấp cơ chế băm mật khẩu an toàn bằng thuật toán BCrypt, có khả năng sinh salt ngẫu nhiên và điều chỉnh độ phức tạp của quá trình mã hóa. Trong hệ thống quản lý giao hàng, thư viện này được sử dụng để mã hóa mật khẩu người dùng trước khi lưu trữ vào cơ sở dữ liệu, giúp bảo vệ thông tin đăng nhập và giảm thiểu nguy cơ tấn công dò mật khẩu hoặc rò rỉ dữ liệu.

2.3.2.10. GOOGLE.ORTOOLS (9.11.4210)

Google.OrTools là bộ thư viện tối ưu hóa toán học do Google phát triển, hỗ trợ giải các bài toán tối ưu tuyến đường, lập lịch và phân công tài nguyên. Trong dự án quản lý giao hàng, OrTools được sử dụng để tính toán và gợi ý tuyến đường giao hàng tối ưu cho tài xế dựa trên khoảng cách, thời gian và số lượng điểm giao. Điều này giúp giảm chi phí nhiên liệu, tiết kiệm thời gian và tăng hiệu suất vận hành.

2.3.2.11. MICROSOFT.ASPNETCORE.AUTHENTICATION.JWTBEARER (8.0.8)

Gói thư viện này được sử dụng để xác thực người dùng thông qua cơ chế JSON Web Token (JWT) trong môi trường ASP.NET Core. Mỗi yêu cầu từ ứng dụng di động khi gửi đến máy chủ đều phải mang theo token hợp lệ để truy cập các tài nguyên được bảo vệ. Việc tích hợp JwtBearer giúp hệ thống đảm bảo an toàn truy cập, quản lý phiên đăng nhập, đồng thời hỗ trợ phân quyền người dùng theo vai trò như quản trị viên, tài xế hoặc nhân viên điều phối.

2.3.2.12. SYSTEM.IDENTITYMODEL.TOKENS.JWT (8.8.0)

System.IdentityModel.Tokens.Jwt là thư viện phục vụ cho việc tạo và xác minh các token JWT. Ở phía máy chủ, hệ thống sử dụng thư viện này để sinh token chứa thông tin định danh người dùng, ký số bằng khóa bảo mật và xác định thời hạn sử dụng. Khi người dùng gửi yêu cầu đến API, token sẽ được giải mã để xác thực danh tính và đảm bảo tính toàn vẹn của dữ liệu truyền qua mạng.

2.3.2.13. MICROSOFT.ENTITYFRAMEWORKCORE.SQLSERVER (8.0.8)

Đây là gói kết nối chính thức giữa Entity Framework Core và cơ sở dữ liệu SQL Server. Hệ thống quản lý giao hàng sử dụng gói này để ánh xạ các thực thể như Đơn hàng, Tài xế, Xe, Hàng hóa và Điểm giao đến các bảng dữ liệu tương ứng. Việc sử dụng Entity Framework Core giúp việc thao tác dữ liệu trở nên đơn giản, thống nhất và giảm thiểu lỗi truy vấn SQL thủ công.

2.3.2.14. MICROSOFT.ENTITYFRAMEWORKCORE.TOOLS (8.0.8)

Gói công cụ này hỗ trợ việc tạo, chỉnh sửa và áp dụng các migration trong quá trình phát triển cơ sở dữ liệu bằng Entity Framework Core. Nhóm phát triển sử dụng nó để đồng bộ hóa cấu trúc dữ liệu giữa môi trường phát triển và triển khai, giúp kiểm soát phiên bản cơ sở dữ liệu, cập nhật lược đồ và quản lý thay đổi một cách an toàn.

2.3.2.15. SWASHBUCKLE.ASPNETCORE (6.6.2)

Swashbuckle.AspNetCore là thư viện giúp tự động sinh tài liệu API theo chuẩn OpenAPI/Swagger cho các ứng dụng ASP.NET Core. Trong dự án quản lý giao hàng, thư viện này được sử dụng để tạo giao diện thử nghiệm API, giúp các lập trình viên kiểm tra trực tiếp các endpoint như đăng nhập, tạo đơn hàng, phân công tài xế hay theo dõi trạng thái giao hàng. Swagger UI còn cho phép chèn JWT vào phần xác thực, giúp kiểm thử các API yêu cầu quyền truy cập dễ dàng và trực quan.

3. CHƯƠNG 3 PHÂN TÍCH THIẾT KẾ HỆ THỐNG

3.1. MÔ TẢ NGHIỆP VỤ

Hệ thống Delivery Management được xây dựng nhằm tái hiện toàn bộ quy trình vận hành của một doanh nghiệp giao hàng nhỏ một cách số hóa và tự động hóa. Khi chưa có hệ thống, mọi công việc

đều được thực hiện thủ công: ghi chép đơn hàng bằng giấy, gọi điện cho tài xế, tính toán lộ trình dựa vào kinh nghiệm, và cập nhật tiến độ qua tin nhắn. Điều đó khiến việc giao hàng trở nên thiếu nhất quán, dễ sai sót và khó kiểm soát. Hệ thống này ra đời để thay đổi toàn bộ điều đó — tạo ra một luồng hoạt động thống nhất, thông minh và phản ánh chính xác thực tế hoạt động của doanh nghiệp.

Ngay từ khâu đầu tiên, chủ doanh nghiệp sẽ tiến hành nhập các dữ liệu nền tảng cần thiết. Đây là bước được xem như “xương sống” của hệ thống, đảm bảo mọi quy trình sau đó được vận hành chính xác. Trong phần quản lý hàng hóa, chủ doanh nghiệp định nghĩa danh sách các sản phẩm thường xuyên giao nhận: từ thực phẩm, vật liệu xây dựng, đến thiết bị điện tử. Mỗi sản phẩm có mã định danh, đơn vị tính, trọng lượng và mô tả chi tiết. Việc quản lý này không chỉ để tạo đơn hàng nhanh chóng mà còn giúp doanh nghiệp dễ theo dõi lượng hàng còn lại, thống kê các mặt hàng được giao nhiều nhất và nhận biết xu hướng kinh doanh theo thời gian.

Sau khi hoàn thiện danh mục hàng hóa, người quản lý tiếp tục nhập thông tin xe vận chuyển. Ở các doanh nghiệp nhỏ, số lượng xe không nhiều nhưng mỗi xe lại có đặc điểm riêng: loại xe, tải trọng, dung tích nhiên liệu, ngày đăng kiểm, và tình trạng hoạt động. Mỗi khi xe được thêm vào hệ thống, chủ doanh nghiệp có thể gán trực tiếp cho một tài xế cụ thể. Hệ thống đảm bảo một nguyên tắc quan trọng — một tài xế chỉ điều khiển một xe tại một thời điểm, và khi có sự thay đổi, liên kết cũ sẽ được tự động gỡ bỏ để tránh trùng lặp. Ngoài ra, phần quản lý xe cũng cho phép ghi nhận lịch bảo dưỡng, giúp doanh nghiệp dễ dàng theo dõi và lên kế hoạch sửa chữa định kỳ.

Một trong những tính năng quan trọng nhất là quản lý điểm giao hàng. Doanh nghiệp thường có hàng chục, thậm chí hàng trăm khách hàng ở các khu vực khác nhau. Mỗi điểm giao không chỉ bao gồm tên người nhận, số điện thoại mà còn có địa chỉ cụ thể. Khi chủ doanh nghiệp chỉ nhập địa chỉ, hệ thống sẽ tự động gọi dịch vụ bản đồ để lấy tọa độ GPS chính xác. Điều này đặc biệt hữu ích trong quá trình tối ưu hóa lộ trình, giúp hệ thống biết chính xác vị trí của từng điểm giao để tính toán đường đi ngắn nhất. Những dữ liệu này được xử lý hoàn toàn tự động thông qua dịch vụ định vị tích hợp với OpenStreetMap.

Sau khi đã có hàng hóa, xe, tài xế và điểm giao, chủ doanh nghiệp bắt đầu tạo đơn hàng. Đây là khâu trung tâm của toàn bộ quy trình. Mỗi đơn hàng bao gồm các thông tin cơ bản như mã đơn, điểm giao, mặt hàng, số lượng, đơn giá và xe thực hiện. Khi đơn hàng được tạo, hệ thống sẽ tự động phân loại trạng thái là “chờ giao”, đồng thời tính toán tổng tiền dựa trên danh sách chi tiết hàng hóa. Mọi thay đổi, dù là thêm hay xóa một sản phẩm, đều được cập nhật tức thời, đảm bảo

tính chính xác tuyệt đối. Điều này giúp chủ doanh nghiệp tránh các lỗi thường gặp như tính sai giá trị đơn hàng hoặc quên cập nhật số lượng.

Khi các đơn hàng đã sẵn sàng, tài xế bắt đầu đảm nhận nhiệm vụ của mình thông qua ứng dụng di động. Ứng dụng được thiết kế với giao diện trực quan, thân thiện, dễ thao tác ngay cả với người không rành công nghệ. Ngay khi đăng nhập, tài xế nhìn thấy danh sách các đơn hàng được giao cho xe của mình, kèm theo thông tin chi tiết về từng đơn. Tài xế có thể xem tên người nhận, địa chỉ, số điện thoại và ghi chú như “liên hệ trước khi đến” hoặc “giao vào giờ hành chính”.

Trước khi khởi hành, tài xế kích hoạt chức năng tối ưu lộ trình. Hệ thống backend xử lý yêu cầu bằng hai giai đoạn. Giai đoạn đầu sử dụng Google OR-Tools để xác định thứ tự các điểm giao sao cho tổng quãng đường ngắn nhất. Giai đoạn tiếp theo sử dụng OSRM để lấy dữ liệu đường bộ thực tế, tính toán thời gian di chuyển chính xác và vẽ tuyến đường trên bản đồ. Kết quả hiển thị trực tiếp trên ứng dụng di động, giúp tài xế dễ dàng hình dung hành trình của mình. Các điểm dừng được đánh số theo thứ tự tối ưu, đường đi được tô màu, và thời gian dự kiến đến từng điểm được ước tính rõ ràng.

Trong suốt quá trình giao hàng, ứng dụng trở thành người bạn đồng hành của tài xế. Tài xế có thể bấm vào từng điểm giao để mở chỉ đường trực tiếp bằng Google Maps. Khi đến nơi, họ nhấn nút “Hoàn thành”, hệ thống ngay lập tức cập nhật trạng thái đơn hàng thành “đã giao”, ghi nhận thời gian thực tế và gửi thông báo cho chủ doanh nghiệp. Ở phía quản lý, chủ doanh nghiệp chỉ cần mở ứng dụng là có thể thấy danh sách đơn hàng đã hoàn thành, các đơn đang giao và vị trí xe trên bản đồ theo thời gian thực.

Đối với doanh nghiệp nhỏ, việc nắm bắt tức thời tình trạng giao hàng mang lại giá trị rất lớn. Chủ doanh nghiệp không còn phải gọi điện cho tài xế để hỏi “đã đến chưa” hay “giao xong chưa”, mà chỉ cần mở ứng dụng để thấy toàn bộ tiến trình. Mọi dữ liệu được lưu trữ đồng bộ, giúp quản lý có thể xem lại lịch sử giao hàng của từng tài xế, từng xe hoặc từng khách hàng. Những thông tin như thời gian giao trung bình, số lượng đơn hoàn thành trong ngày, hoặc tỷ lệ giao đúng giờ đều được thống kê tự động.

Bên cạnh đó, hệ thống còn cung cấp tính năng báo cáo tổng hợp giúp doanh nghiệp phân tích hiệu quả vận hành. Sau mỗi ngày hoặc mỗi tuần, hệ thống tự động tổng hợp các chỉ số: số đơn hàng giao thành công, tổng quãng đường di chuyển, lượng nhiên liệu ước tính tiêu thụ và thời gian trung bình cho mỗi chuyến. Những dữ liệu này được thể hiện dưới dạng biểu đồ trực quan, giúp người quản lý dễ dàng nhận biết tài xế nào hoạt động hiệu quả, xe nào cần bảo dưỡng hoặc tuyến đường

nào thường mất nhiều thời gian. Những phân tích này không chỉ giúp cải thiện năng suất mà còn hỗ trợ doanh nghiệp ra quyết định về điều phối nhân lực và kế hoạch vận hành trong tương lai.

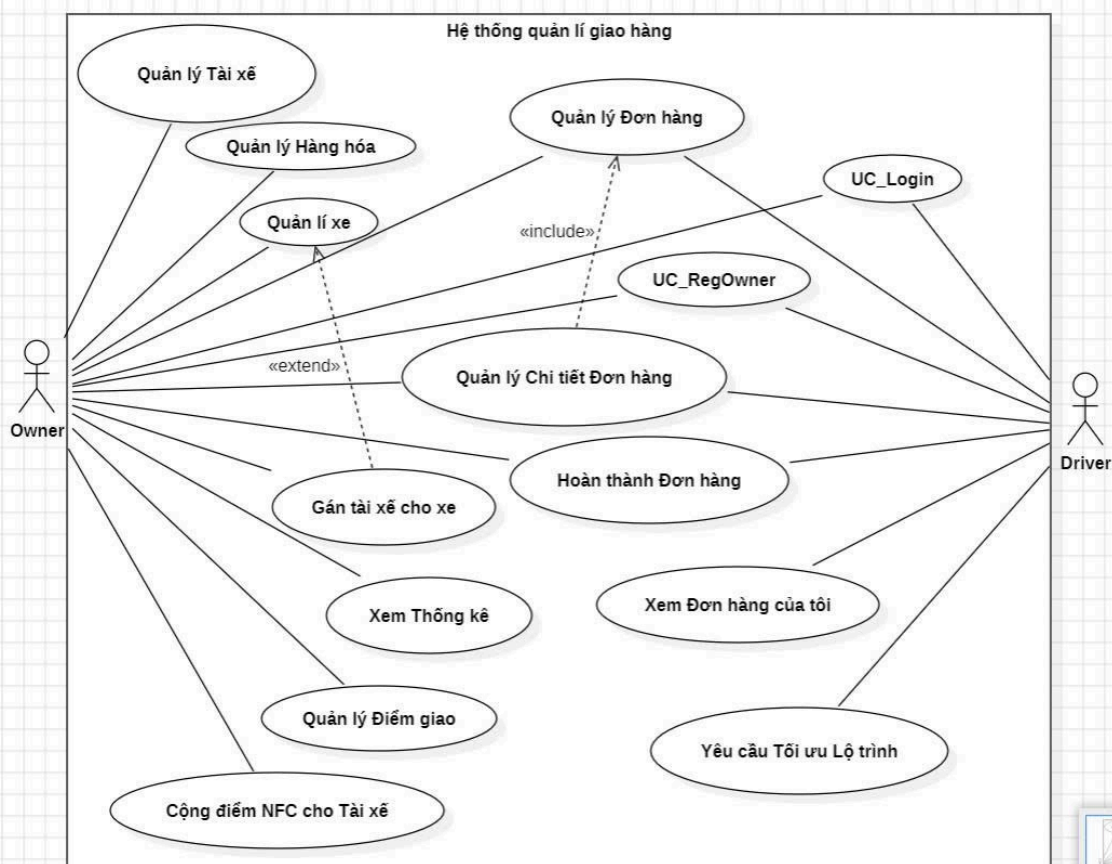
Về mặt kỹ thuật, hệ thống được xây dựng với kiến trúc full-stack hiện đại, tách biệt rõ giữa giao diện người dùng và xử lý dữ liệu. Phần frontend sử dụng Flutter, giúp ứng dụng hoạt động mượt mà trên cả Android và iOS, còn backend sử dụng ASP.NET Core, chịu trách nhiệm xử lý logic nghiệp vụ, xác thực người dùng bằng JWT và kết nối cơ sở dữ liệu SQL Server. Việc sử dụng công nghệ này mang lại hiệu năng cao, bảo mật tốt và dễ dàng mở rộng trong tương lai.

Một vai trò quan trọng khác là người quản trị kỹ thuật. Họ đảm bảo hệ thống luôn hoạt động ổn định, theo dõi hiệu suất máy chủ, cập nhật phần mềm và sao lưu dữ liệu định kỳ. Dù không tham gia trực tiếp vào quy trình giao hàng, nhưng quản trị viên đóng vai trò thầm lặng trong việc duy trì hệ thống. Mỗi tuần, họ kiểm tra nhật ký hoạt động, đảm bảo các API phản hồi đúng và không có lỗi bảo mật. Trong trường hợp có sự cố, họ là người chịu trách nhiệm khôi phục dữ liệu và đảm bảo dịch vụ không bị gián đoạn.

Toàn bộ hoạt động của hệ thống vận hành như một guồng máy thống nhất. Mỗi bộ phận – từ chủ doanh nghiệp, tài xế đến quản trị viên – đều đóng vai trò quan trọng trong một chuỗi khép kín. Dữ liệu được nhập vào, xử lý, phản hồi và lưu trữ liên tục, tạo thành một vòng lặp hoàn chỉnh: nhập dữ liệu gốc, tạo đơn hàng, tối ưu lộ trình, thực thi, xác nhận, và báo cáo. Mỗi bước đều được tự động hóa, giảm tối đa sự can thiệp thủ công và rủi ro sai sót.

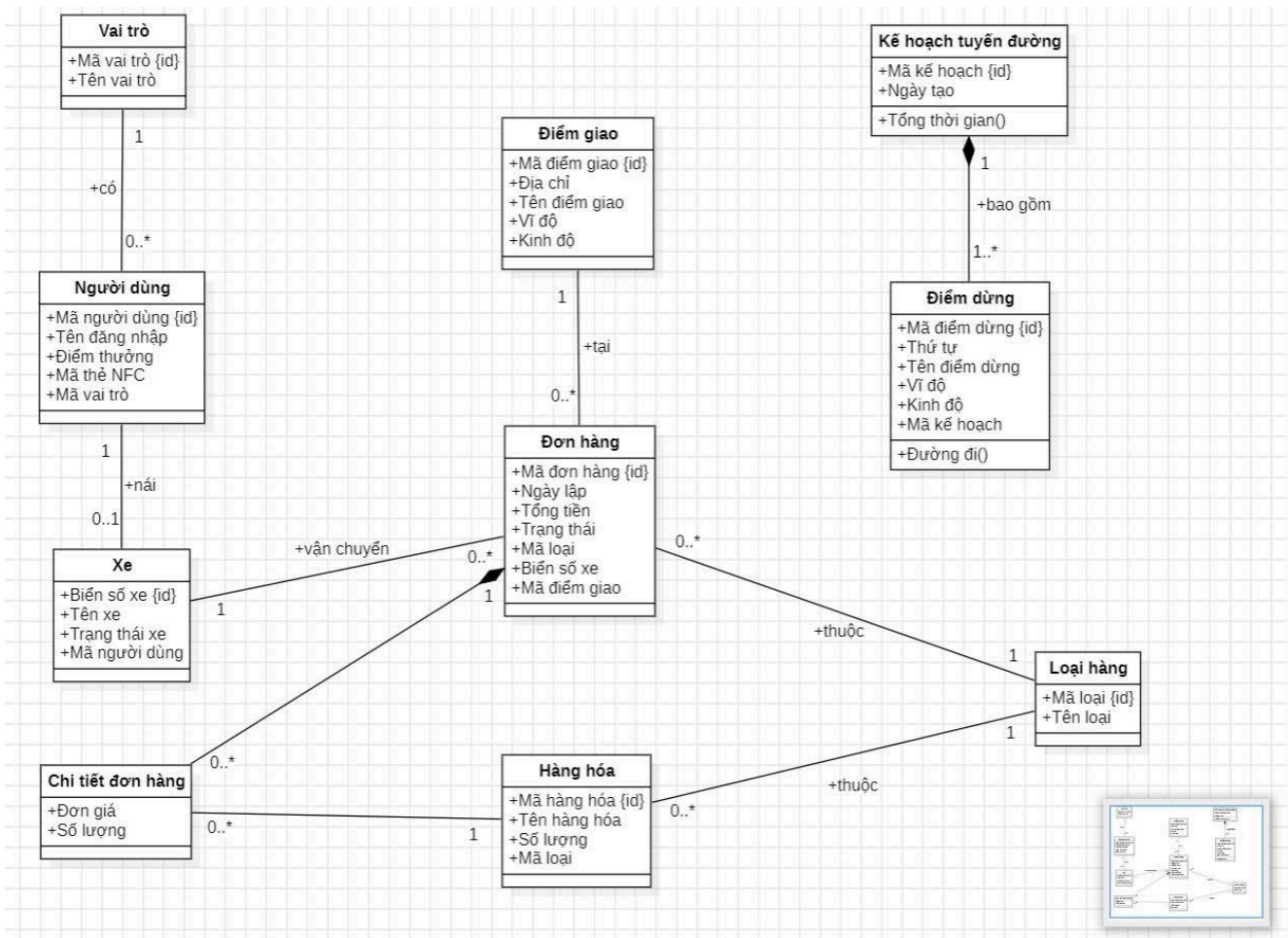
Khi nhìn tổng thể, Delivery Management không chỉ là một ứng dụng giao hàng, mà là một hệ thống quản lý vận hành thông minh. Nó giúp các doanh nghiệp nhỏ – vốn không có điều kiện đầu tư hệ thống quản trị phức tạp – có thể vận hành chuyên nghiệp như một công ty logistics thực thụ. Mỗi chuyến giao hàng đều được số hóa, mỗi tài xế đều có lịch trình rõ ràng, và mỗi quyết định của chủ doanh nghiệp đều dựa trên dữ liệu thực tế. Đây không chỉ là phần mềm, mà là một bước tiến trong hành trình chuyển đổi số của các doanh nghiệp Việt Nam đang phát triển mạnh mẽ trong thời đại công nghệ.

3.2. SƠ ĐỒ USECASE



Hình 3.1 : Sơ đồ UseCase

3.3. MÔ HÌNH QUAN HỆ



Hình 3.2 Sơ đồ Class

3.3.1. MÔ TẢ CÁC QUAN HỆ

Sau khi phân tích mô hình thực thể và các nghiệp vụ cốt lõi trong hệ thống, ta tiến hành chuyển sang mô hình quan hệ của hệ thống quản lý giao hàng Delivery Management. Mô hình này bao gồm 10 bảng dữ liệu chính, phản ánh đầy đủ các hoạt động từ quản lý hàng hóa, đơn hàng, xe vận chuyển, tài xế cho đến việc tối ưu hóa lộ trình giao hàng.

Bảng LOAIHANG(MALOAİ, TENLOAI, SL):

Mỗi loại hàng được định danh bằng mã loại hàng duy nhất MALOAİ. Thông tin bao gồm tên loại TENLOAI và số lượng SL cho biết tổng số hàng hóa thuộc nhóm này. Mã loại hàng được quản lý theo quy tắc không trùng lặp, tên loại không được để trống và số lượng là kiểu số nguyên. Một loại hàng có thể chứa nhiều hàng hóa khác nhau, thể hiện mối quan hệ một-nhiều (1-n) giữa LOAIHANG và HANGHOA.

Bảng **HANGHOA**(MAHH, TENHH, SL, MALOAI):

Mỗi hàng hóa trong hệ thống có mã hàng hóa MAHH duy nhất, tên hàng TENHH, số lượng tồn SL và thuộc về một loại hàng cụ thể thông qua khóa ngoại MALOAI. Mã hàng hóa là khóa chính, không trùng lặp; tên hàng không được để trống; và MALOAI liên kết đến bảng LOAIHANG. Khi một loại hàng bị xóa, các hàng hóa thuộc loại đó sẽ bị ràng buộc không thể xóa nếu đang tồn tại trong đơn hàng. Một hàng hóa có thể xuất hiện trong nhiều chi tiết đơn hàng khác nhau, thể hiện mối quan hệ một-nhiều giữa **HANGHOA** và **CT_DONHANG**.

Bảng **DIEMGIAO**(D_DD, TEN, VITRI, Lat, Lng):

Bảng này lưu thông tin về các điểm giao hàng hay khách hàng nhận hàng. Mỗi điểm giao được định danh bằng mã D_DD duy nhất, có tên điểm TEN, địa chỉ VITRI và tọa độ vị trí Lat, Lng. Hai thuộc tính Lat và Lng giúp hệ thống có thể tính toán lộ trình giao hàng tối ưu thông qua dịch vụ bản đồ. Một điểm giao có thể liên quan đến nhiều đơn hàng được giao tới cùng một địa chỉ, tạo nên mối quan hệ một-nhiều giữa **DIEMGIAO** và **DONHANG**.

Bảng **XE**(BS_XE, TENXE, TT_XE, UserId):

Đây là bảng quản lý các phương tiện vận chuyển. Mỗi xe có biển số BS_XE duy nhất, tên xe TENXE, trạng thái hoạt động TT_XE và tài xế được gán UserId. Một xe có thể được gán cho một tài xế duy nhất tại một thời điểm. Khi thay đổi, hệ thống tự động gỡ tài xế khỏi xe cũ để đảm bảo tính toàn vẹn. Một xe có thể thực hiện nhiều đơn hàng trong cùng một ngày, thể hiện mối quan hệ một-nhiều giữa **XE** và **DONHANG**.

Bảng **USER**(UserId, Username, PasswordHash, FullName, PhoneNumber, CCCD, RoleId, CreatedAt):

Bảng này lưu thông tin người dùng trong hệ thống, bao gồm hai vai trò chính là Chủ doanh nghiệp (Owner) và Tài xế (Driver). Mỗi người dùng được định danh bằng UserId duy nhất, có tên đăng nhập Username, mật khẩu được mã hóa PasswordHash, họ tên, số điện thoại, và mã định danh cá nhân CCCD. Trường RoleId xác định quyền hạn của người dùng trong hệ thống. Một người dùng có thể được gán cho một xe duy nhất, thể hiện mối quan hệ một-một giữa **USER** và **XE**.

Bảng **ROLES**(RoleId, RoleName):

Bảng này xác định các vai trò của người dùng trong hệ thống, bao gồm Owner và Driver. Mỗi vai trò được định danh bằng RoleId và tên vai trò RoleName. Một vai trò có thể được gán cho nhiều người dùng khác nhau, thể hiện mối quan hệ một-nhiều giữa **ROLES** và **USER**.

Bảng **DONHANG**(MADON, BS_XE, D_DD, MALOAI, NGAYLAP, NGAYGIAO, TONGTIEN, TRANGTHAI, ServiceMinutes, WindowStart, WindowEnd):

Bảng này là trung tâm của hệ thống, lưu trữ thông tin các đơn hàng. Mỗi đơn hàng có mã MADON

duy nhất, liên kết với xe vận chuyển thông qua BS_XE, điểm giao hàng thông qua D_DD và loại hàng thông qua MALOAI. Các trường NGAYLAP và NGAYGIAO lưu thời gian lập và hoàn tất đơn hàng, TONGTIEN là tổng giá trị hàng hóa, TRANGTHAI thể hiện trạng thái đơn (ví dụ: chờ giao, đang giao, hoàn thành). Ngoài ra còn có các thuộc tính ServiceMinutes, WindowStart, WindowEnd dùng để hỗ trợ tính toán thời gian phục vụ và khung giờ giao hàng trong quá trình tối ưu lộ trình. Một đơn hàng có thể có nhiều chi tiết hàng hóa, thể hiện mối quan hệ một-nhiều giữa DONHANG và CT_DONHANG.

Bảng CT_DONHANG(MAHH, MADON, SL, DONGIA):

Bảng chi tiết đơn hàng chứa thông tin từng mặt hàng trong mỗi đơn. Mỗi bản ghi là sự kết hợp giữa mã hàng hóa MAHH và mã đơn hàng MADON. Các trường SL và DONGIA cho biết số lượng và đơn giá của sản phẩm đó trong đơn hàng. Cặp MAHH và MADON là khóa chính kép, đảm bảo mỗi hàng hóa chỉ xuất hiện một lần trong một đơn hàng. Một đơn hàng có thể có nhiều chi tiết, còn một hàng hóa có thể xuất hiện trong nhiều đơn hàng khác nhau.

Bảng ROUTEPLANS(Id, CreatedAt, Note, TotalSeconds):

Bảng này lưu thông tin về các kế hoạch lộ trình đã được tối ưu. Mỗi kế hoạch có mã định danh Id duy nhất, thời gian tạo CreatedAt, ghi chú Note và tổng thời gian di chuyển TotalSeconds. Một kế hoạch lộ trình có thể chứa nhiều điểm dừng, thể hiện mối quan hệ một-nhiều giữa ROUTEPLANS và ROUTESTOPS.

Bảng ROUTESTOPS(Id, RoutePlanId, Name, Order, Lat, Lng, EtaEpoch, EtaIso, Polyline):

Bảng ROUTESTOPS lưu chi tiết từng điểm dừng trong lộ trình. Mỗi điểm có thứ tự Order, tên Name, tọa độ Lat và Lng, thời gian dự kiến đến EtaEpoch hoặc EtaIso, và đường đi Polyline. Các điểm này liên kết với bảng ROUTEPLANS thông qua khóa ngoại RoutePlanId. Mỗi kế hoạch lộ trình có thể có nhiều điểm dừng, và mỗi điểm dừng thuộc về một kế hoạch duy nhất.

Tổng kết lại, các bảng trong hệ thống có mối liên hệ chặt chẽ, tạo thành một mô hình dữ liệu thống nhất: LOAIHANG – HANGHOA – CT_DONHANG – DONHANG phản ánh chuỗi quản lý hàng hóa và đơn hàng; DIEMGIAO – DONHANG – XE thể hiện quy trình giao nhận thực tế; USERS – XE – DONHANG phản ánh mối liên hệ giữa tài xế và hoạt động giao hàng; ROUTEPLANS – ROUTESTOPS mô tả quy trình tối ưu và thực thi lộ trình. Tất cả tạo thành một hệ thống quản lý logic, đầy đủ và linh hoạt, đáp ứng tốt yêu cầu vận hành thực tế của doanh nghiệp nhỏ trong lĩnh vực giao vận.

3.3.2. LƯỢC ĐỒ QUAN HỆ CƠ SỞ DỮ LIỆU

LOAIHANG(**MALOAI**, TENLOAI, SL).

HANGHOA(**MAHH**, TENHH, SL, MALOAI).

DIEMGIAO(**D_DD**, TEN, VITRI, Lat, Lng).

XE(**BS_XE**, TENXE, TT_XE, UserId).

USERS(**UserId**, Username, PasswordHash, FullName, PhoneNumber, CCCD, RoleId, CreatedAt, Scores).

ROLES(**RoleId**, RoleName).

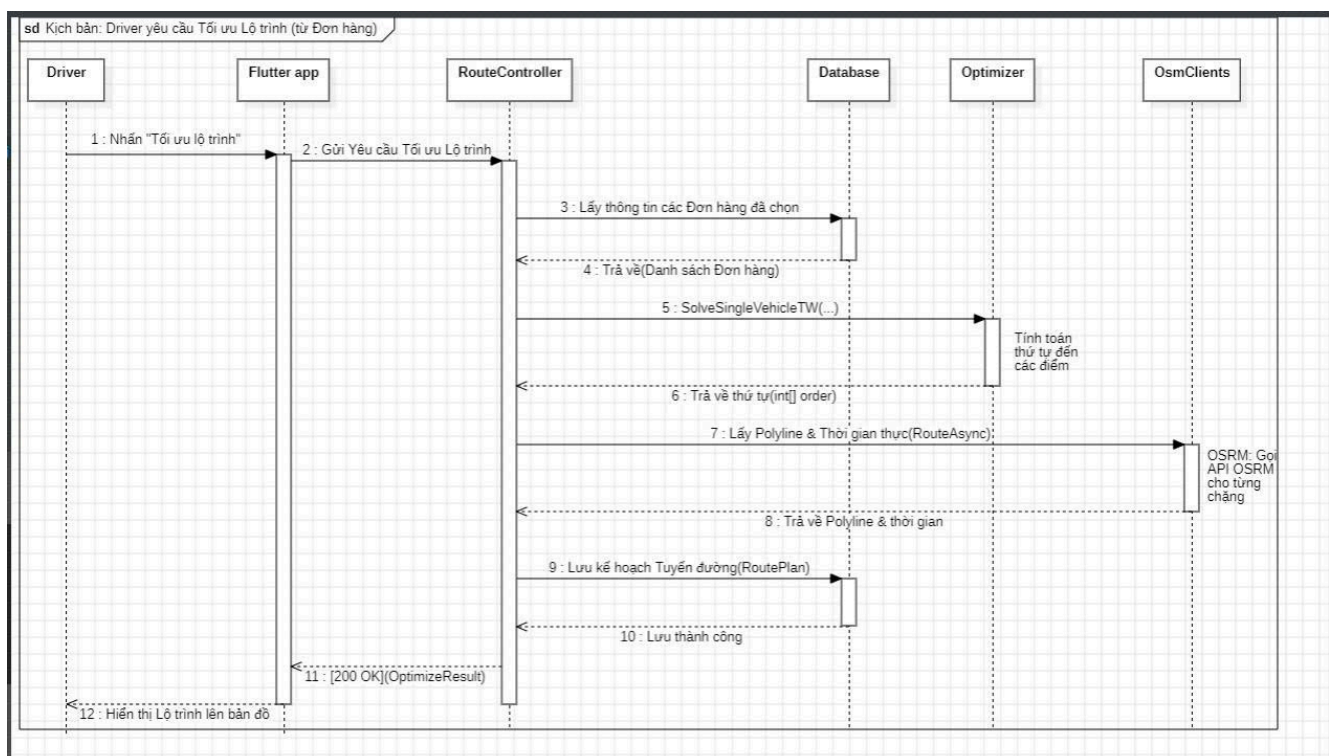
DONHANG(**MADON**, BS_XE, D_DD, MALOAI, NGAYLAP, NGAYGIAO, TONGTIEN, TRANGTHAI, ServiceMinutes, WindowStart, WindowEnd).

CT_DONHANG(**MAHH**, MADON, SL, DONGIA).

ROUTEPLANS(**Id**, CreatedAt, Note, TotalSeconds).

ROUTESTOPS(**Id**, RoutePlanId, Name, [Order], Lat, Lng, EtaEpoch, EtaIso, Polyline).

3.4. SƠ ĐỒ SEQUENCE



4. CHƯƠNG 4 KẾT QUẢ THỰC NGHIỆM

4.1. MÔ HÌNH HỆ THỐNG

4.1.1. MÔ HÌNH CẤU TRÚC HỆ THỐNG

Hệ thống Delivery Management được thiết kế dựa trên mô hình Client – Server ba lớp (Three-Tier Architecture), với mục tiêu đảm bảo sự tách biệt rõ ràng giữa giao diện người dùng, xử lý nghiệp vụ và lưu trữ dữ liệu, giúp hệ thống dễ bảo trì, mở rộng và nâng cấp về sau.

Ở lớp Frontend (Client), ứng dụng được phát triển bằng Flutter (ngôn ngữ Dart), chạy trên nền tảng di động Android. Giao diện được chia thành hai vai trò chính: Owner (Chủ doanh nghiệp) và Driver (Tài xế).

Giao diện Owner giúp quản lý danh sách đơn hàng, xe, tài xế, hàng hóa và theo dõi tình trạng giao hàng theo thời gian thực.

Giao diện Driver hỗ trợ tài xế xem các đơn hàng được giao, định vị điểm giao hàng trên bản đồ, tối ưu lộ trình, xác nhận hoàn thành đơn và cập nhật trạng thái giao hàng.

Toàn bộ thao tác từ ứng dụng di động đều được truyền đến Backend Server, là API RESTful được xây dựng bằng ASP.NET Core Web API. Lớp này đóng vai trò trung gian xử lý nghiệp vụ — nhận yêu cầu từ Client, thực hiện các logic kiểm tra, xác thực người dùng thông qua JWT Token, truy xuất dữ liệu từ cơ sở dữ liệu và trả kết quả dạng JSON về ứng dụng.

Ví dụ, khi Owner tạo mới một đơn hàng, hệ thống sẽ gửi yêu cầu POST đến /api/DonHang, sau đó API sẽ ghi dữ liệu xuống bảng DONHANG trong cơ sở dữ liệu SQL Server. Khi Driver bấm “Hoàn thành giao hàng”, một yêu cầu PATCH hoặc PUT sẽ được gửi để cập nhật trạng thái đơn hàng sang “HOANTHANH”.

Lớp Data Layer (Cơ sở dữ liệu) được triển khai trên Microsoft SQL Server, quản lý toàn bộ dữ liệu của hệ thống thông qua Entity Framework Core (EF Core). Các bảng chính bao gồm:

Users (thông tin người dùng, vai trò và tài xế),

Roles (phân quyền Owner/Driver),

XE (danh sách xe và tình trạng xe),

DONHANG (thông tin đơn hàng, tổng tiền, trạng thái, thời gian lập và giao),

CT_DONHANG (chi tiết hàng hóa trong từng đơn hàng),

DIEMGIAO (thông tin tọa độ, địa chỉ các điểm giao),

HANGHOA, LOAIHANG (danh mục sản phẩm và nhóm hàng),

ROUTEPLANS và ROUTESTOPS (dữ liệu tối ưu lộ trình và danh sách điểm dừng giao hàng).

Các mối quan hệ giữa bảng được thiết lập bằng khóa ngoại (Foreign Key), đảm bảo tính toàn vẹn dữ liệu.

Ví dụ:

Một Xe (XE) có thể được gán cho nhiều Đơn hàng (DONHANG).

Mỗi Đơn hàng có thể chứa nhiều Chi tiết đơn hàng (CT_DONHANG), và mỗi Chi tiết liên kết đến một Hàng hóa (HANGHOA) cụ thể.

Mỗi Điểm giao (DIEMGIAO) có thể thuộc nhiều đơn hàng khác nhau, nhưng chỉ lưu một lần để tránh trùng lặp địa điểm.

Các Lộ trình (RoutePlan) có thể bao gồm nhiều Điểm dừng (RouteStop), mỗi điểm lưu tọa độ, thứ tự dừng và mã hóa Polyline đường đi.

Tổng thể, mô hình cấu trúc hệ thống vận hành như một chuỗi khép kín:

Ứng dụng di động gửi yêu cầu → API xử lý nghiệp vụ → SQL Server lưu và trả dữ liệu → API phản hồi → Ứng dụng hiển thị kết quả.

Cấu trúc này giúp hệ thống hoạt động ổn định, phản hồi nhanh, và dễ dàng mở rộng các chức năng như tối ưu lộ trình, nhận diện vật thể, hay thông báo thời gian thực trong tương lai

4.1.2. MÔ HÌNH TỔNG THỂ HỆ THỐNG

Hệ thống Delivery Management được xây dựng như một mạng lưới thống nhất, nơi các thành phần phần mềm – từ người dùng đến máy chủ và cơ sở dữ liệu – phối hợp chặt chẽ để đảm bảo quy trình giao hàng diễn ra trơn tru và minh bạch. Nếu hình dung bằng lời, mô hình tổng thể này giống như một dòng chảy thông tin khép kín: dữ liệu được tạo ra từ chủ doanh nghiệp, xử lý bởi hệ thống trung tâm, và phản hồi ngược lại cho tài xế theo thời gian thực.

Khi chủ doanh nghiệp (Owner) mở ứng dụng, họ có thể tạo đơn hàng mới, khai báo thông tin hàng hóa, chọn xe và tài xế phù hợp. Các thông tin này được gửi tới máy chủ trung tâm (ASP.NET Core Web API) để xử lý và lưu trữ vào cơ sở dữ liệu SQL Server. Tại đây, hệ thống không chỉ lưu dữ

liệu đơn thuần mà còn kích hoạt quy trình tối ưu hóa lộ trình – sử dụng các thuật toán định tuyến thông minh từ Google OR-Tools kết hợp với dữ liệu bản đồ của OSRM API. Kết quả là một lộ trình tối ưu được tính toán, đảm bảo các điểm giao được sắp xếp hợp lý, tiết kiệm thời gian và chi phí vận chuyển.

Sau khi hệ thống xử lý xong, ứng dụng của tài xế (Driver App) sẽ nhận được thông tin cập nhật tức thì: danh sách các điểm giao, thứ tự dừng, thời gian dự kiến đến từng địa điểm và bản đồ đường đi cụ thể. Tài xế có thể mở bản đồ, theo hướng dẫn đến từng điểm giao, xác nhận giao hàng thành công, và gửi phản hồi ngược về máy chủ. Mỗi khi tài xế bấm “Hoàn thành giao hàng”, ứng dụng lập tức gửi yêu cầu đến API `/Driver/complete/$maDon`, để hệ thống cập nhật trạng thái đơn hàng, thời gian giao và quãng đường đã di chuyển.

Trong toàn bộ quy trình, mỗi thành phần của hệ thống đều đóng vai trò rõ ràng:

Ứng dụng Flutter là giao diện người dùng – nơi nhập, xem và tương tác dữ liệu.

API Web Server (ASP.NET Core) là “bộ não”, xử lý nghiệp vụ, tính toán, xác thực và giao tiếp giữa các bên.

SQL Server Database là “trái tim dữ liệu”, lưu trữ toàn bộ thông tin về người dùng, xe, đơn hàng, điểm giao và lộ trình.

Nhìn một cách tổng quan, mô hình tổng thể của hệ thống có thể hình dung như sau:

Chủ doanh nghiệp tạo đơn hàng → API xử lý và lưu trữ → Tối ưu hóa lộ trình → Tài xế nhận nhiệm vụ và xác nhận hoàn thành → Cập nhật trạng thái trở lại hệ thống → Chủ doanh nghiệp theo dõi báo cáo.

Dòng chảy này liên tục và khép kín, đảm bảo mọi thay đổi từ phía người dùng đều được ghi nhận và phản ánh trong toàn bộ hệ thống. Nhờ cấu trúc này, ứng dụng không chỉ phục vụ việc giao hàng thông thường mà còn tạo nền tảng cho các tính năng mở rộng trong tương lai như: giám sát vị trí GPS theo thời gian thực, nhận diện hàng hóa bằng AI, hoặc phân tích hiệu suất giao hàng dựa trên dữ liệu lịch sử.

Nói cách khác, mô hình tổng thể hệ thống giống như một chuỗi vận hành thông minh: mọi mắt xích – từ dữ liệu, xử lý đến hiển thị – đều được kết nối chặt chẽ, giúp doanh nghiệp nhỏ quản lý hoạt động giao hàng một cách hiệu quả, trực quan và hiện đại

4.2. GIAO DIỆN API

The screenshot displays the Swagger UI for the 'Smart Inventory API' (version 1, OAS 3.0). The interface is organized into sections for different API endpoints, each with a list of HTTP methods and their corresponding URLs. The endpoints are categorized as follows:

- Auth**
 - POST `/api/Auth/register-first-owner`
 - POST `/api/Auth/login`
- CtDonHang**
 - GET `/api/CtDonHang`
 - POST `/api/CtDonHang`
 - GET `/api/CtDonHang/{madon}/{mahh}`
 - PUT `/api/CtDonHang/{madon}/{mahh}`
 - DELETE `/api/CtDonHang/{madon}/{mahh}`
- DonHang**
 - GET `/api/DonHang`
 - POST `/api/DonHang`
 - GET `/api/DonHang/{id}`
 - PUT `/api/DonHang/{id}`
 - DELETE `/api/DonHang/{id}`
 - POST `/api/DonHang/{id}/complete`
- Driver**
 - GET `/api/Driver/my-deliveries`
 - POST `/api/Driver/complete/{maDon}`
- HangHoa**
 - GET `/api/HangHoa`
 - POST `/api/HangHoa`
 - GET `/api/HangHoa/{id}`
 - PUT `/api/HangHoa/{id}`

Each endpoint entry includes a color-coded button for the HTTP method (e.g., green for POST, blue for GET, orange for PUT, red for DELETE) and a lock icon indicating authentication requirements. The interface also features a top navigation bar with the Swagger logo and a dropdown menu to select the API definition.

Hình 4.3 Giao diện API phần 1

DELETE	/api/HangHoa/{id}	🔒	▼
Owner ^			
GET	/api/Owner/dashboard	🔒	▼
POST	/api/Owner/add-vehicle	🔒	▼
POST	/api/Owner/add-score	🔒	▼
Route ^			
POST	/api/Route/optimize	🔒	▼
POST	/api/Route/optimize-from-orders	🔒	▼
User ^			
GET	/api/User/drivers	🔒	▼
POST	/api/User/create-driver	🔒	▼

Xe ^			
GET	/api/Xe	🔒	▼
POST	/api/Xe	🔒	▼
GET	/api/Xe/{id}	📱🔒	▼
DELETE	/api/Xe/{id}	🔒	▼
PUT	/api/Xe/{bsxe}	🔒	▼
PUT	/api/Xe/{bsxe}/assign-driver	🔒	▼

Schemas ^			
AssignDriverDto >			
CtDonHang >			
CtDonHangCreateDto >			
CtDonHangReadDto >			
CtDonHangUpdateDto >			
DiemGiao >			
DonHang >			
DonHangCreateDto >			
DonHangReadDto >			
DonHangUpdateDto >			

Hình 4.3 Giao diện API phần 2

HangHoa >
HangHoaCreateDto >
HangHoaReadDto >
HangHoaUpdateDto >
LoaiHang >
LoginRequestDto >
NfcScoreRequestDto >
OptimizeFromOrdersRequest >
OptimizePoint >
OptimizeRequest >
ProblemDetails >
RegisterRequestDto >
Role >
User >
Xe >
XeCreateDto >
XeReadDto >
XeUpdateDto >

Hình 4.3 Giao diện API phần 3

5. CHƯƠNG 5: KẾT LUẬN VÀ KIẾN NGHỊ

5.1. KẾT LUẬN CHUNG

Hiểu được quy trình vận hành và những thách thức trong việc quản lý giao nhận hàng hóa của các doanh nghiệp nhỏ sở hữu đội xe riêng. Có khả năng ứng dụng tốt các công nghệ phát triển ứng dụng di động và xây dựng hệ thống Backend. Áp dụng thành công Flutter với ngôn ngữ Dart để phát triển ứng dụng di động đa nền tảng, đáp ứng nhu cầu nghiệp vụ riêng biệt cho cả Chủ doanh nghiệp và Tài xế. Xây dựng Backend bằng C# (ASP.NET Core) theo mô hình API và kiến trúc Clean Architecture để đảm bảo tính linh hoạt và khả năng bảo trì.

Đứng trước nhu cầu cấp thiết về việc tối ưu hóa quy trình điều phối, theo dõi và quản lý giao vận cho các doanh nghiệp nhỏ có **n** xe và **m** tài xế, nhóm đã quyết định xây dựng hệ thống Delivery

Management – Ứng dụng quản lý giao hàng cho doanh nghiệp nhỏ – một nền tảng di động toàn diện được thiết kế để số hóa và đơn giản hóa các hoạt động logistics hàng ngày. Hệ thống bao gồm hai giao diện chính, phục vụ cho hai vai trò cốt lõi: Chủ doanh nghiệp (Owner) và Tài xế (Driver).

Qua quá trình thực hiện đồ án, nhóm đã nhận thức sâu sắc về tiềm năng của công nghệ di động và định vị GPS trong việc giải quyết các bài toán logistics phức tạp. Delivery Management không chỉ là công cụ giúp Chủ doanh nghiệp quản lý đơn hàng, phương tiện và nhân sự hiệu quả hơn, mà còn là trợ thủ đắc lực cho Tài xế trên mọi nẻo đường, giúp tối ưu lộ trình và đơn giản hóa quy trình giao nhận.

5.2. KẾT QUẢ ĐẠT ĐƯỢC

Tổng quan: Đã xây dựng thành công một hệ thống ứng dụng di động (Flutter) và backend API (ASP.NET Core) hoàn chỉnh, phân chia rõ ràng hai vai trò người dùng (Owner, Driver) với các chức năng nghiệp vụ cốt lõi, số hóa quy trình quản lý và thực thi giao vận.

Đối với Chủ doanh nghiệp (Owner):

Quản lý Dữ liệu Gốc:

Cung cấp giao diện CRUD (Thêm-Xóa-Sửa-Xem) đầy đủ và an toàn cho Hàng hóa (HangHoa), bao gồm kiểm tra ràng buộc không cho xóa nếu hàng hóa đang được sử dụng.

Cung cấp giao diện CRUD đầy đủ cho Xe (Xe), cho phép quản lý thông tin cơ bản (biển số, tên xe, trạng thái) và thực hiện nghiệp vụ quan trọng: gán Tài xế (Driver) cho xe, đảm bảo một tài xế chỉ lái một xe tại một thời điểm. Hệ thống cũng ngăn chặn xóa xe nếu đang được gán cho đơn hàng. (Lưu ý: Chức năng quản lý chi tiết như giấy tờ, lịch bảo dưỡng chưa được triển khai trong phiên bản hiện tại).

Cung cấp giao diện CRUD đầy đủ cho Điểm giao (DiemGiao), tích hợp tính năng Geocoding tự động thông minh: hệ thống tự lấy và lưu tọa độ (Lat/Lng) từ địa chỉ nếu người dùng không nhập thủ công, sử dụng dịch vụ OpenStreetMap Nominatim.

Quản lý Đơn hàng:

Cho phép tạo Đơn hàng (DonHang) mới, gán đơn hàng cho một Điểm giao cụ thể. (Lưu ý: Việc gán Xe cho Đơn hàng khi tạo ban đầu chưa hoạt động do thiếu trường dữ liệu, cần được xử lý ở bước tối ưu hoặc cập nhật sau).

Cho phép truy cập chi tiết đơn hàng để thêm/sửa/xóa các mặt hàng (CtDonHang) cần giao, bao gồm số lượng và đơn giá.

Backend tự động tính toán và cập nhật tổng tiền (TONGTIEN) của DonHang một cách chính xác mỗi khi chi tiết đơn hàng thay đổi, đảm bảo tính toàn vẹn dữ liệu.

Đối với Tài xế (Driver):

Cung cấp giao diện xem danh sách các đơn hàng (DonHang) được gán cho xe của mình (trạng thái "Chờ giao").

Cho phép kích hoạt chức năng Tối ưu Lộ trình từ tab Bản đồ.

Hiển thị lộ trình đã tối ưu trực quan trên bản đồ (MapTab), bao gồm thứ tự các điểm dừng, đường đi (polyline), và ETA (thời gian đến dự kiến) được tính toán dựa trên dữ liệu đường bộ thực tế từ OSRM.

Hiển thị danh sách các điểm giao (DeliveriesTab) có thể được sắp xếp lại theo đúng thứ tự tối ưu đã tính toán, đồng bộ với bản đồ. (Lưu ý: Chức năng xem chi tiết hàng hóa cần giao tại mỗi điểm từ tab này chưa được triển khai).

Tích hợp nút "Bắt đầu" để mở ứng dụng Google Maps chỉ đường đến điểm giao tiếp theo.

Cung cấp nút "Hoàn thành" sau khi giao hàng xong. Khi nhấn, ứng dụng gọi API để cập nhật trạng thái DonHang thành "HOANTHANH" và ghi nhận thời gian giao thực tế (NGAYGIAO) trên server. Đơn hàng hoàn thành sẽ tự động ẩn khỏi danh sách chờ giao. (Lưu ý: Quy trình xác nhận lên hàng/đã đến và thông báo tự động cho Owner chưa được triển khai).

Tính năng Kỹ thuật Cốt lõi:

Tối ưu hóa lộ trình (Hybrid Approach): Triển khai thành công thuật toán tối ưu lộ trình thông minh, kết hợp Google OR-Tools (để tìm thứ tự tối ưu nhanh dựa trên ma trận thời gian đường chim bay) và OSRM API (để lấy dữ liệu đường bộ thực tế - polyline, ETA - cho lộ trình đã chọn), cân bằng hiệu quả giữa tốc độ và độ chính xác của kết quả cuối cùng.

Giao diện & Trải nghiệm Người dùng: Xây dựng giao diện ứng dụng di động Flutter rõ ràng, phân chia theo tab chức năng phù hợp với từng vai trò (Owner, Driver). Sử dụng Provider để quản lý và chia sẻ trạng thái (đặc biệt là RouteProvider giữa 2 tab của Driver).

Hiệu năng: Backend API được thiết kế chú trọng hiệu năng (ví dụ: AsNoTracking cho truy vấn đọc, logic UpdateDonHangTongTienAsync hiệu quả). Luồng cập nhật UI sau khi hoàn thành đơn hàng được xử lý tốt (optimistic update).

Kiến trúc & Bảo mật:

Backend API (ASP.NET Core) có cấu trúc phân lớp rõ ràng (Controller, Service, Domain, DTO, Data), dễ quản lý và mở rộng. Áp dụng xác thực JWT và phân quyền theo Role ([Authorize]) để đảm bảo an toàn cho các endpoint nghiệp vụ.

Hosting: [Swagger UI](#).

5.3. KẾT QUẢ CHƯA ĐẠT ĐƯỢC

Do thời gian nghiên cứu và nguồn lực có hạn, hệ thống Delivery Management – Ứng dụng quản lý giao hàng cho doanh nghiệp nhỏ vẫn còn một số hạn chế cần khắc phục để tiến tới mức độ hoàn thiện cao hơn trong môi trường vận hành thực tế.

Trước hết, các chức năng nâng cao dựa trên AI/ML chưa được tích hợp. Ý tưởng nhận diện vật thể

bằng camera để hỗ trợ kiểm kê, nhận dạng nhanh hàng hóa theo mã, nhãn hoặc đặc trưng hình ảnh chưa được triển khai. Kiểm tra cấu hình ứng dụng di động cho thấy dự án chưa bổ sung các thư viện học máy như TensorFlow Lite cũng như các quyền camera phục vụ pipeline nhận diện. Ở chiều hỗ trợ thao tác rảnh tay cho tài xế, các năng lực chuyển văn bản thành giọng nói và nhận dạng giọng nói (TTS/STT) cũng chưa xuất hiện trong cấu hình gói phụ thuộc; vì vậy, luồng điều khiển bằng giọng nói hoặc phát hướng dẫn tự động vẫn chưa sẵn sàng để dùng trong thực tế.

Tiếp theo, quy trình Bằng chứng giao hàng (Proof of Delivery) chưa có. Luồng hiện tại kết thúc khi tài xế nhấn nút hoàn thành; hệ thống cập nhật trạng thái và thời gian giao xong nhưng không thu thập được bằng chứng đi kèm để xử lý tranh chấp. Các tính năng còn thiếu bao gồm chụp ảnh kiện hàng tại điểm giao, lấy chữ ký điện tử của người nhận, cũng như cơ chế lưu trữ và liên kết bằng chứng với đơn hàng trong cơ sở dữ liệu. Việc thiếu các bằng chứng này khiến khâu hậu kiểm, đối soát và giải quyết khiếu nại chưa đạt mức tin cậy cần thiết trong nghiệp vụ logistics.

Thứ ba, hệ thống thông báo thời gian thực chưa được triển khai, dẫn đến trải nghiệm “kéo dề làm mới” ở cả phía chủ doanh nghiệp và tài xế. Chủ doanh nghiệp chỉ thấy trạng thái đơn thay đổi sau khi làm mới danh sách; tài xế cũng cần thao tác thủ công để nhận các đơn mới. Việc chưa tích hợp kênh đẩy thời gian thực (ví dụ SignalR ở phía máy chủ hoặc Firebase Cloud Messaging ở ứng dụng di động) làm giảm tính kịp thời của thông tin, nhất là trong bối cảnh vận hành nhiều xe, nhiều đơn trong một ngày.

Thứ tư, cần định hình một số hiểu lầm liên quan đến tối ưu hóa lộ trình để đánh giá đúng năng lực hiện tại của hệ thống. Về ràng buộc thời gian, thuật toán đã xét đến thời gian phục vụ tại điểm giao và cửa sổ thời gian giao hàng; các tham số này được chuẩn hóa ở phía dịch vụ và gán vào mô hình tối ưu để ràng buộc lịch trình. Như vậy, nhận định “chưa xem xét thời gian chờ và time window” là không còn chính xác. Tuy nhiên, nhận định hệ thống chưa xét đến giao thông thời gian thực là hợp lý: dữ liệu tuyến hiện dùng có tính chất điển hình theo lịch sử chứ chưa phản ánh tức thời tình trạng kẹt xe theo thời gian thực. Điều này khiến ETA và thứ tự tối ưu đôi lúc lệch so với thực địa trong giờ cao điểm.

Bên cạnh đó, cảm nhận “tối ưu chậm” chủ yếu đến từ điểm nghẽn thu thập dữ liệu đầu vào chứ không nằm ở công cụ tối ưu. Trước khi tối ưu, ứng dụng phải lấy chi tiết từng đơn và xây dựng danh sách điểm; hiện tượng gọi lặp $N+1$ lần đến máy chủ cho N đơn hàng tạo ra độ trễ tổng đáng kể. Thuật toán tối ưu chạy nhanh trên ma trận sơ bộ, nhưng toàn bộ thời gian chờ gom dữ liệu và gọi định tuyến từng chặng mới là phần chi phối trải nghiệm. Khi chưa khắc phục được vấn đề gom dữ liệu theo lô, cache kết quả địa lý hay hợp nhất lời gọi API, người dùng vẫn sẽ thấy quá trình chuẩn bị lộ trình bị chậm.

Ngoài bốn nhóm vấn đề chính nêu trên, hệ thống vẫn còn một số điểm “chưa đạt” ở mức nền tảng. Chế độ ngoại tuyến cho tài xế chưa được triển khai, nên khi mất kết nối mạng tài xế không thể xác nhận giao xong và đồng bộ sau đó. Hệ thống thông tin quản trị chưa có bảng điều khiển thống kê tổng hợp để hỗ trợ ra quyết định nhanh về hiệu suất xe và tài xế. Khía cạnh vận hành – bảo mật như chuẩn hóa chính sách mật khẩu, quản lý vòng đời phiên đăng nhập, giới hạn tốc độ gọi API, nhật ký truy cập và cảnh báo an toàn chưa được hoàn thiện đầy đủ. Cuối cùng, quy trình kiểm thử và quan sát hệ thống (test tự động, theo dõi hiệu năng, log hợp nhất) còn mỏng, khiến việc phát hiện sớm lỗi và suy giảm hiệu năng trong các bản phát hành có thể gặp khó khăn.

Tóm lại, hệ thống hiện đã đáp ứng tốt các chức năng cốt lõi như quản lý dữ liệu gốc, tạo và theo dõi đơn hàng, gán xe – tài xế, tối ưu lộ trình có ràng buộc thời gian và xác nhận hoàn thành. Tuy nhiên, để đạt chất lượng vận hành gần với chuẩn doanh nghiệp, dự án cần tiếp tục hoàn thiện các lớp chức năng nâng cao (AI/ML và bằng chứng giao hàng), bổ sung hạ tầng thời gian thực và tối ưu pipeline dữ liệu trước tối ưu; song song với đó là nâng cấp trải nghiệm người dùng, tăng cường

bảo mật, khả dụng ngoại tuyến và năng lực quan sát – kiểm thử. Những hạng mục này chính là trọng tâm cho các giai đoạn phát triển tiếp theo.

5.4. HƯỚNG PHÁT TRIỂN

Với những giới hạn hiện tại của hệ thống, đề phần mềm Delivery Management – Ứng dụng quản lý giao hàng cho doanh nghiệp nhỏ có thể hoàn thiện hơn và đáp ứng tốt hơn nhu cầu thực tế của các doanh nghiệp, trong thời gian tới nhóm dự kiến sẽ tập trung phát triển và tối ưu ở những khía cạnh sau:

Trước hết, nhóm sẽ hoàn thiện các chức năng còn thiếu để hệ thống trở nên toàn diện hơn. Một trong những hướng phát triển chính là tích hợp công nghệ Nhận diện vật thể (Object Detection) nhằm hỗ trợ kiểm kê hàng hóa tự động, giúp việc nhập – xuất kho trở nên chính xác và nhanh chóng hơn. Bên cạnh đó, hệ thống sẽ được mở rộng tích hợp công nghệ Chuyển văn bản thành giọng nói (TTS) và Nhận dạng giọng nói (STT) vào quy trình làm việc của tài xế, giúp việc thao tác trong khi lái xe trở nên thuận tiện và an toàn hơn. Ngoài ra, nhóm dự kiến bổ sung chức năng “Bằng chứng giao hàng” (Proof of Delivery), cho phép tài xế chụp ảnh hoặc ký điện tử để xác nhận đơn hàng giao thành công, đồng thời mở rộng hệ thống thông báo đa dạng hơn, bao gồm thông báo theo thời gian thực về tình trạng đơn hàng, xe và lộ trình.

Về mặt tối ưu hóa và nâng cao hiệu năng, nhóm tập trung cải thiện thuật toán tối ưu lộ trình hiện tại. Hệ thống sẽ xem xét bổ sung thêm các yếu tố thực tế như tình hình giao thông, thời gian phục vụ tại từng điểm và độ ưu tiên của đơn hàng để đưa ra lộ trình ngắn và hiệu quả nhất. Ngoài ra, nhóm sẽ tiến hành tối ưu tốc độ tải dữ liệu, cải thiện phản hồi giao diện, đặc biệt hướng đến các thiết bị di động có cấu hình thấp nhằm đảm bảo trải nghiệm mượt mà. Trải nghiệm người dùng (UI/UX) cũng sẽ được nâng cấp cho cả hai vai trò — chủ doanh nghiệp và tài xế — dựa trên phản hồi thực tế thu thập trong quá trình sử dụng. Song song đó, các công cụ tìm kiếm và lọc dữ liệu sẽ được cải thiện để chủ doanh nghiệp dễ dàng tra cứu đơn hàng, xe, tài xế hoặc khách hàng. Một tính năng khác cũng được xem xét là xây dựng bảng điều khiển thống kê (Dashboard) cho chủ doanh nghiệp, cung cấp báo cáo tổng quan về hiệu suất giao hàng, quãng đường di chuyển, tần suất đơn hàng và năng suất của từng tài xế.

Về phần mở rộng chức năng, nhóm sẽ phát triển giao diện hoặc ứng dụng dành riêng cho khách hàng, cho phép họ theo dõi tình trạng đơn hàng, nhận thông báo và phản hồi trực tiếp. Đồng thời, hệ thống cũng sẽ nghiên cứu và triển khai chế độ làm việc ngoại tuyến (Offline Mode) cho tài xế, giúp họ có thể xác nhận giao hàng và lưu trữ dữ liệu tạm thời ngay cả khi không có kết nối mạng, sau đó tự động đồng bộ khi có mạng trở lại.

Cuối cùng, nhóm sẽ tiếp tục chú trọng đến vấn đề bảo mật và độ tin cậy của dữ liệu. Cụ thể, nhóm sẽ tăng cường bảo vệ hệ thống API thông qua các cơ chế xác thực, mã hóa và kiểm soát truy cập nâng cao. Ngoài ra, nhóm dự kiến hợp tác cùng các doanh nghiệp thực tế để kiểm thử phần mềm trong môi trường vận hành thật, từ đó đánh giá độ ổn định, khả năng mở rộng, và hoàn thiện hệ thống nhằm đáp ứng tốt hơn nhu cầu thực tế của thị trường giao vận hiện nay.

